

Siêu hướng dẫn sử dụng Olympia HTML5

Bộ điều khiển đồ hoạ và âm thanh chạy trên trình duyệt cho gameshow "Đường lên đỉnh Olympia": cách dùng, cách sửa, cách mở rộng và cách hoạt động bên trong.



Bản 30/09/2026

Thư mục dự án: C:\inetpub\wwwroot\2015_Retro\

Đã đối chiếu với mã nguồn trong thư mục dự án ngày 30/09/2026 (xem mục 1.4).

Mục lục

Chương 1. Olympia HTML5 là gì?

1.1 Định nghĩa · 1.2 Nguồn gốc · 1.3 Thành phần · 1.4 Nguồn của tài liệu này

Chương 2. Hướng dẫn sử dụng và điều khiển

2.1 Khởi chạy · 2.2 Màn đồ hoạ · 2.3 Màn quản lý · 2.4 Bảng phím chung · 2.5 Luồng từng vòng · 2.6 Luật tính điểm đang cài

Chương 3. Hướng dẫn chỉnh sửa tính năng

3.1 Chỉnh đồ hoạ · 3.2 Chỉnh animation · 3.3 Chỉnh cơ chế có sẵn của vòng thi (mức 1: không đụng logic, mức 2: có đụng logic)

Chương 4. Hướng dẫn thêm tính năng

4.1 Thêm đồ hoạ và animation · 4.2 Thêm cơ chế mới cho vòng thi · 4.3 Thêm vòng thi: ví dụ Câu hỏi phụ · 4.4 Quy tắc an toàn · 4.5 Kiểm tra và lỗi thường gặp

Chương 5. Cơ chế hoạt động chi tiết

5.1 Kiến trúc hai trang · 5.2 Thứ tự nạp file · 5.3 Trạng thái và đồng bộ · 5.4 Vòng đời một phím · 5.5 Bộ đếm bước của bốn vòng · 5.6 Chuyển vòng · 5.7 Âm thanh 7 kênh · 5.8 Đồng hồ chung · 5.9 Khởi động trang · 5.10 Đọc XML

Chương 6. So sánh với WWTBAM HTML5 Controller

6.1 Hai bộ khác nhau ở đâu · 6.2 Bảng đối chiếu file · 6.3 Luồng từ phím tới lệnh · 6.4 Hình ảnh so sánh

Chương 7. Một số lưu ý khác khi chạy

7.1 Máy chủ và trình duyệt · 7.2 Tư liệu và dữ liệu · 7.3 Khi điều khiển

Phụ lục. Bảng tra nhanh

A.1 Bảng tra nhanh phím · A.2 Muốn làm gì thì mở file nào · A.3 Danh sách ví dụ trong tài liệu

Chương 1

Olympia HTML5 là gì?

Cho mọi người đọc. Mục 1.4 nói rõ tài liệu này dựa trên nguồn nào.

1.1 Định nghĩa

Olympia HTML5 (trong mã nguồn gọi là "Olympia Controller") là bộ điều khiển đồ hoạ và âm thanh chạy trên trình duyệt, dùng để dựng lại gameshow "Đường lên đỉnh Olympia" (VTV) cho một buổi thi tổ chức tại chỗ. Một buổi thi có 4 thí sinh và 4 vòng: **Khởi động** (KĐ), **Vượt chướng ngại vật** (VCNV), **Tăng tốc** (TT) và **Về đích** (VĐ).

Vòng thứ 5, **Câu hỏi phụ**, đã được dành chỗ: phím F5, mã vòng `tb`. Vòng này chưa có mã; bấm F5 hiện chỉ chuyển sang một vòng rỗng. Mục 4.3 dùng nó làm ví dụ cho việc thêm một vòng thi.

Bộ điều khiển viết bằng HTML, CSS và JavaScript thuần, không thư viện, không có bước build. Chép thư mục vào IIS là chạy.

1.2 Nguồn gốc

Thư mục dự án là `C:\inetpub\wwwroot\2015_Retro\`. Dự án thay thế một bộ **WWTBAM** ("Ai là triệu phú") HTML5 Controller từng nằm ở cùng thư mục, vào ngày 11/09/2026. Bộ WWTBAM gốc được giữ lại trong thư mục con `WWTBAM Original Project`.

Ngày 23/09/2026, toàn bộ mã JavaScript được viết lại theo đúng lối của bộ WWTBAM gốc (bản mà người dùng gọi là Olga V2.5): hàm toàn cục, trạng thái là biến phẳng trong `window.GameVariables`, mỗi vòng một bộ đếm bước, mọi phím nằm trong một file `KeyboardController.js`. Mục đích là để người đã quen sửa bộ WWTBAM đọc và sửa được Olympia ngay. Chương 6 so sánh hai bộ.

Đồ hoạ lấy từ bộ ảnh trong thư mục `O24/`, mỗi vòng một thư mục con: `Warm-up`, `Beat the Obstacle`, `Accelerate`, `Finish Line`. Âm thanh nằm trong `Sounds/`, tên ASCII theo vòng (tiền tố `kd_`, `vcnv_`, `tt_`, `vd_`). Các file này được chép và đổi tên từ một bộ âm thanh gốc có tên tiếng Việt.

1.3 Thành phần

Hệ thống gồm hai trang web chạy song song trên cùng một máy.

Trang	Vai trò	Ai dùng
`default.html`	**Màn đồ hoạ.** Đưa lên máy chiếu hoặc màn phát sóng. Là trang duy nhất phát âm thanh và giữ trạng thái thật (authority).	Khán giả, thí sinh
`control.html`	**Màn quản lý.** Hiển thị trạng thái, gửi phím và lệnh sang `default.html` (remote). Chỉ nạp `Variables.js`, `StateSync.js`, `KeyboardController.js`, `ControlPanel.js`.	Người điều khiển

Các thư mục và file còn lại:

Đường dẫn	Chứa gì
`Scripts/`	Toàn bộ JavaScript. Bảng thứ tự nạp và vai trò từng file nằm ở mục 5.2.
`Styles/olympia.css`, `Styles/control.css`	Giao diện của màn đồ hoạ và màn quản lý.

`Questions/`	`players.xml`, `khoidong.xml`, `vcnv.xml`, `tangtoc.xml`, `vedich.xml`. Mỗi file có chú thích cách điền ở đầu. Ảnh và video của đề cũng để ở đây (ví dụ `tt1.png`, `tt4.mp4`, `vcnv_picture.png`).
`Sounds/`	Âm thanh, tên ASCII theo vòng.
`O24/`	Ảnh đồ hoạ, mỗi vòng một thư mục con; `O24/background.png` là nền, `O24/glare.png` là ánh loé.
`Fonts/`	Font dùng trên màn đồ hoạ.
`web.config`	Cấu hình IIS: trang mặc định, kiểu MIME cho font, âm thanh, video; tắt bộ nhớ đệm.
`WWTBAM Original Project/`	Bộ WWTBAM HTML5 Controller gốc, giữ để tham chiếu.



Hình 1. Thẻ tên bốn vòng: `O24/Warm-up/text.png`, `O24/Beat the Obstacle/text.png`, `O24/Accelerate/text.png`, `O24/Finish Line/text.png`.

1.4 Nguồn của tài liệu này

Bản trước của tài liệu được viết từ tư liệu dự án mà chưa đối chiếu với mã. Bản này đã được đối chiếu với mã nguồn trong thư mục dự án ngày 30/09/2026 và sửa theo mã. Những chỗ sửa chính: bốn sơ đồ bộ đếm ở mục 5.5, chỗ hiện cảnh báo "trang đồ hoạ thứ hai" (mục 5.1), cách gom bản chụp trạng thái (mục 5.3), mẫu animation D (mục 3.2), và luật CSS còn thiếu trong ví dụ dải GIẢI LAO (mục 4.1).

- Khung nét đứt "ẢNH CHỤP MÀN HÌNH #N" là chỗ để người dùng tự chèn ảnh chụp sau. Các ảnh đồ hoạ (thẻ tên, thanh câu hỏi...) là ảnh thật lấy từ dự án.
- Đoạn mã có ghi "trích từ mã" là chép từ file nguồn. Đoạn có ghi "ví dụ" là mã minh hoạ viết theo đúng quy ước của dự án; chưa có trong dự án cho tới khi bạn tự thêm.
- Phụ lục A.3 liệt kê mọi ví dụ trong tài liệu.
- Nếu sau này mã và tài liệu khác nhau, tin mã nguồn.

Chương 2

Hướng dẫn sử dụng và điều khiển

Cho người điều khiển buổi thi. Không cần biết lập trình.

2.1 Khởi chạy

1. Chạy dự án qua IIS (hoặc một máy chủ web khác). Không mở file bằng đường dẫn `file://`, vì trình duyệt sẽ chặn việc đọc file XML.
2. Mở `http://localhost/2015_Retro/` trên máy chiếu hoặc màn phát. Trang mặc định là `default.html`.
3. Mở `http://localhost/2015_Retro/control.html` trên màn của người điều khiển, **cùng một trình duyệt, cùng một máy**.
4. Đặt màn phát ở 1920×1080, zoom trình duyệt 100 %, bấm F11 để toàn màn hình. Sân khấu cố định 1920×1080 px và không tự co giãn.
5. Bấm một phím bất kỳ trên trang đồ hoạ trước khi thi, để trình duyệt cho phép phát âm thanh.

2.2 Màn đồ hoạ

Lúc mới mở, màn đồ hoạ chỉ có nền (Hình 2). Nền luôn hiện. Đồ hoạ của từng vòng hiện và tắt theo phím người điều khiển bấm; mục 2.5 mô tả từng bước.

ẢNH CHỤP MÀN HÌNH #1

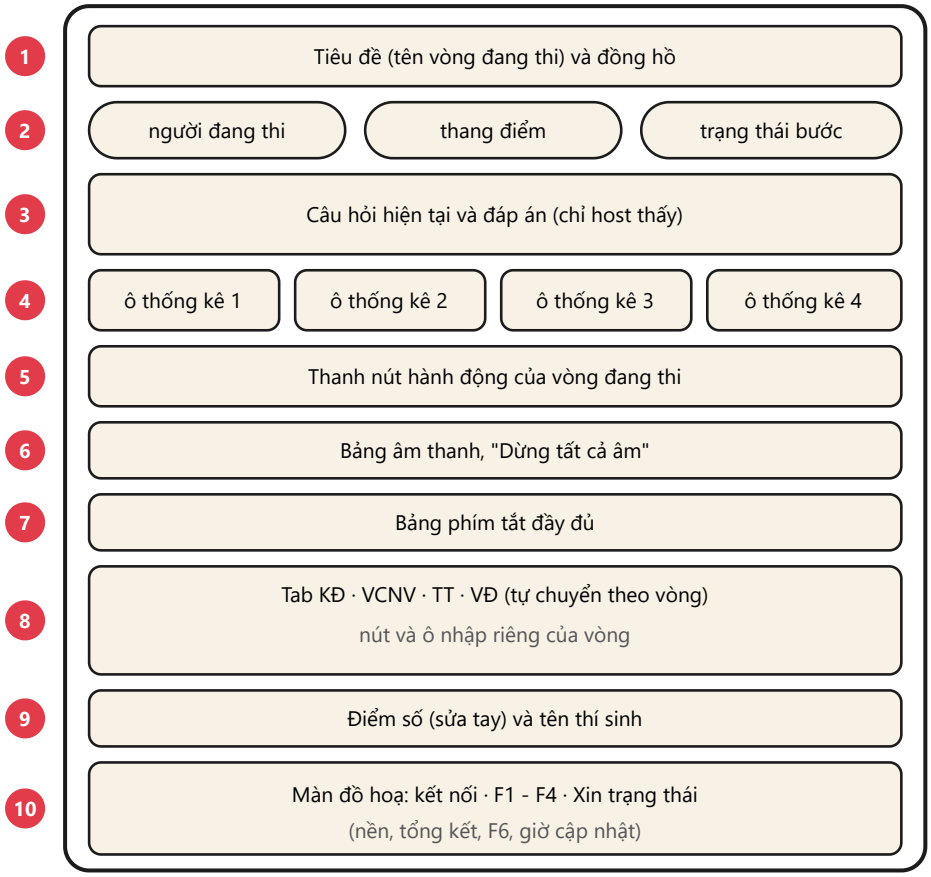
Màn đồ hoạ lúc mới mở, chỉ có nền.

Hình 2. Màn đồ hoạ (`default.html`) lúc mới mở.

Tuỳ vòng, màn đồ hoạ có thể hiện: thẻ tên vòng, 4 thanh tên thí sinh kèm điểm, thanh câu hỏi, bảng ô chữ và ảnh 5 mảnh (VCNV), khung câu hỏi ảnh hoặc video (TT), màn chọn gói và nhóm câu hỏi (VĐ), cây đáp án (VCNV và TT), bảng tổng kết điểm (phím M) và bảng kết quả (phím F6). Nếu một tab `default.html` thứ hai được mở, tab đó hiện dòng cảnh báo đỏ ngay trên màn đồ hoạ.

2.3 Màn quản lý

Màn quản lý (`control.html`) không xử lý phím tại chỗ. Phím bấm ở đây được chuyển nguyên sang trang đồ hoạ, và màn quản lý vẽ lại từ bản chụp trạng thái mà trang đồ hoạ gửi về. Hình 3 đánh số các vùng theo thứ tự từ trên xuống.



Hình 3. Thứ tự các vùng của màn quản lý (chỉ thể hiện thứ tự, không theo kích thước thật).

1. **Tiêu đề** (tên vòng đang thi) và **đồng hồ** đang chạy.
2. Ba **chip trạng thái**: người đang thi, thang điểm, trạng thái bước.
3. **Câu hỏi hiện tại và đáp án**. Chỉ người điều khiển thấy.
4. **Bốn ô thống kê** theo vòng.
5. **Thanh nút hành động** của vòng đang thi. Nút ứng với phím nên bấm tiếp được làm nổi.
6. **Bảng âm thanh**: phát chồng lên nhau và không đổi trạng thái vòng; có nút "Dừng tất cả âm".
7. **Bảng phím tắt** đầy đủ.
8. **Tab theo vòng**, tự chuyển theo vòng đang thi (chi tiết bên dưới).
9. **Điểm số** (sửa tay điểm từng ghế) và **tên thí sinh** (đặt tên, nút "Về tên trong players.xml"; tên gõ tay được nhớ trên máy).
10. Khu **Màn đồ hoạ**: trạng thái kết nối (dựa vào nhịp tim của trang đồ hoạ), nút F1 đến F4, nút "Xin trạng thái", tình trạng nền, tổng kết, F6.

Nhãn "KĐ" và "Loại CNV" trên bảng điểm chỉ hiện trong vòng tương ứng.



Hình 4. Màn quản lý toàn trang, vòng Khởi động.

- **Tab theo vòng**

- **KĐ**: đặt lại cả vòng hoặc chỉ lượt chung, xoá chuông, đánh dấu ghế đã thi.
- **VCNV**: ô nhập đáp án của 4 thí sinh cho hàng đang mở, kèm nút "Ghế N đúng" (tự đánh dấu khi đáp án gõ khớp đáp án trong XML). Còn có "Ai đúng chướng ngại vật", "Tất cả người xin đều sai, loại" (không hỏi xác nhận), "Khôi phục quyền thi cho ghế…", "Mở hết ảnh + đáp án" và đặt lại.
- **TT**: ô đáp án và ô thời gian (giây, ví dụ 1.25) cho 4 ghế, tự lưu khi rời ô, tự đánh dấu đúng khi khớp; nút "Ghế N đúng" trong từng dòng; "Chạy lại câu…"; đặt lại.
- **VĐ**: thứ tự thi dạng "Tên A --> Tên B --> Tên C --> Tên D" (người đã thi có dấu ✓); "Đổi thứ tự thi…"; "Chạy lại lượt của ghế…"; đặt lại.



Hình 5. Tab Tăng tốc trên màn quản lý.

2.4 Bảng phím chung

Phím	Tác dụng
F1 F2 F3 F4 F5	Chuyển vòng: KĐ, VCNV, TT, VĐ, Câu hỏi phụ. F5 chưa có nội dung.
M	Tổng kết điểm: mở (điểm thấp nhất trước; bằng điểm thì ghế số nhỏ trước), người kế, đóng. Bị từ chối khi đồ hoạ vòng còn trên màn.
F6	Bảng kết quả hiện tại, bật hoặc tắt. Khi bảng mở, mọi phím khác bị khoá, trừ F6 và dấu chấm.
. (dấu chấm)	Phát tiếng "ô trống" `blank.ogg`.
0	Ẩn hoặc hiện dải thanh tên thí sinh (trừ lúc VCNV đang xét người xin trả lời CNV, khi đó 0 là "tất cả sai").
Z / X	+10 / −5 cho người đang hoạt động (người trên sân khấu hoặc người bấm chuông đầu tiên). Dự phòng.
1-4 khi đang ở F5	Bật hoặc tắt cờ bấm chuông của ghế (thử nghiệm). Ở bốn vòng chính, các phím này thuộc về vòng.

Không giữ phím: phím lặp bị bỏ qua. Phím gõ trong ô nhập liệu không được tính là phím điều khiển. Phím riêng của từng vòng nằm ở mục 2.5; Phụ lục A gom tất cả vào một bảng.

ẢNH CHỤP MÀN HÌNH #9

Bảng tổng kết điểm (phím M).

Hình 6. Bảng tổng kết điểm.

ẢNH CHỤP MÀN HÌNH #10

Bảng kết quả F6.

Hình 7. Bảng kết quả F6.

2.5 Luồng từng vòng

Mỗi bảng đọc từ trên xuống: bấm phím ở cột trái thì xảy ra việc ở cột giữa, kèm âm thanh ở cột phải (nếu có). Tên âm thanh là tên file trong `Sounds/`.

- **Khởi động, lượt riêng**

Mỗi thí sinh thi 6 câu, lần lượt theo số ghế.

Phím	Việc xảy ra	Âm thanh

←	Thẻ tên vòng hiện.	`kd_start`
←	Thẻ tên tắt; thí sinh kế lên sân khấu (thanh tên chuyển đỏ).	`kd_ready`
←	Thanh câu hỏi (có dải đồng hồ) phóng to vào; 4 thanh tên hiện lần lượt.	`kd_open_question`
→	Câu 1 hiện. Nhạc nền bắt đầu và lặp đến câu cuối.	`kd_60s`
Space	Đồng hồ đếm 3-2-1.	
↑	Đúng, +10 điểm.	`kd_correct`
↓	Sai, không trừ điểm.	`kd_wrong`
→	Sang câu kế. Ở câu 6, nhạc nền nhảy tới phần kết.	
→ sau câu 6	Đồ hoạ tắt. Một giây sau phát âm thanh hoàn thành.	`kd_complete`
←	Lặp lại từ bước "thí sinh kế lên sân khấu".	

ẢNH CHỤP MÀN HÌNH #3

Khởi động: thanh câu hỏi và 4 thanh tên, một thí sinh đang bấm chuông.

Hình 8. Vòng Khởi động: thanh câu hỏi, 4 thanh tên.

- **Khởi động, lượt chung**

Sau thí sinh thứ 4 là lượt chung, 12 câu, giành quyền bằng chuông.

Phím	Việc xảy ra	Âm thanh
←	Chuẩn bị lượt chung (không có thẻ tên).	`kd_ready`
←	Thanh câu hỏi không có dải đồng hồ phóng to vào; 4 thanh tên hiện lần lượt.	`kd_open_question`
→	Câu 1 hiện, nhạc nền bắt đầu.	`kd_60s`
1-4	Ghế bấm chuông. Người đầu tiên khoá câu, thanh tên chuyển đỏ.	`kd_buzz`
Space	Đồng hồ 3-2-1 (có hoặc không có người bấm).	
↑	Đúng: +10 cho người bấm. Không ai bấm thì bỏ qua.	`kd_correct`

↓	Sai: −5 cho người bấm. Không ai bấm thì chỉ phát tiếng. Mỗi câu chỉ chấm một lần.	`kd_wrong`
Backspace	Xoá lượt bấm chuông nhầm, chuông mở lại.	
→ sau câu 12	Đồ hoạ tắt, một giây sau âm thanh hoàn thành.	`kd_complete`
←	Sang VCNV (như F2).	

- **Vượt chướng ngại vật**

Bảng có 4 hàng ngang, ô trung tâm và ảnh 5 mảnh. Người điều khiển mở hàng theo thứ tự tuỳ ý. Hàng đúng thì mở một mảnh ảnh.

Phím	Việc xảy ra	Âm thanh
← hoặc F2	Thẻ tên vòng hiện.	`vcnv_start`
←	Bảng ô chữ, ảnh và thanh câu hỏi hiện.	`vcnv_open_board`
1-4	Chọn hàng (hàng chưa mở). Ô tròn của hàng chuyển xanh sáng.	`vcnv_select_row`
5	Chọn câu ô trung tâm. Chỉ chạy khi 4 hàng đã hỏi và CNV chưa giải.	`vcnv_select_row`
U	Huỷ chọn hàng, chỉ khi chưa hiện câu.	
→	Hiện câu hỏi.	`vcnv_open_question`
→	Thanh 15 giây chạy.	`vcnv_15s`
1-4	Đánh dấu hoặc bỏ đánh dấu ghế trả lời đúng (từ lúc hiện câu tới lúc chấm). Chỉ đánh dấu; điểm cộng lúc chấm.	
→	Dừng thanh nếu còn chạy; đồ hoạ VCNV ẩn, cây đáp án hiện.	`vcnv_open_answers`
→	Chấm. Nhánh sai chuyển xám. Hàng đúng hiện chữ, hàng sai xám. Mỗi ghế đúng +10 (ô trung tâm cũng +10).	`vcnv_row_correct` / `vcnv_row_wrong`
→	Cất cây đáp án, đồ hoạ VCNV trở lại.	
→	Mở mảnh ảnh nếu hàng đúng; về bảng để chọn hàng kế.	`vcnv_open_picture`

ẢNH CHỤP MÀN HÌNH #4

VCNV: ô chữ, ảnh và thanh câu hỏi, có tên người xin trả lời CNV.

Hình 9. Vòng VCNV: ô chữ, ảnh, thanh câu hỏi.

ẢNH CHỤP MÀN HÌNH #5

Cây đáp án VCNV sau khi chấm.

Hình 10. Cây đáp án VCNV sau khi chấm.

Giải chướng ngại vật (CNV)

Bốn phím Q W E R ứng với ghế 1 đến 4. Có thể xin trả lời ở mọi bước của hàng, sau khi đã mở ít nhất một hàng.

Phím	Việc xảy ra	Âm thanh
Q W E R	Ghế xin trả lời CNV. Luồng hàng bị khoá ngay (thanh 15 giây nếu đang chạy vẫn chạy tiếp), tên ghế hiện dưới thanh câu hỏi. Ghế khác vẫn xin thêm được.	`vcnv_signal`

Bấm lại phím của ghế đã xin (hoặc 1-4)	Ghế đó đúng. Điểm theo số hàng đã mở: 1 hàng 60, 2 hàng 50, 3 hàng 40, 4 hàng 30; sau khi mở ô trung tâm 20. Mở toàn bộ ảnh và chữ. Ghế chưa xin thì không chọn được.	`vcnv_solved`
Backspace	Bỏ người xin cuối cùng; hết người xin thì quay lại bước hàng đang dở.	
0	Tất cả người xin đều sai. Họ bị loại khỏi phần còn lại của vòng (mất cả quyền trả lời hàng); dấu "đúng" của họ ở hàng đang dở bị gỡ. Quay lại bước hàng đang dở.	`vcnv_row_wrong`

Khi cả 5 câu đã hỏi mà chưa ai giải: ở bảng, → mở 15 giây chót trên thanh câu hỏi trống; hết giờ thì bấm → để mở toàn bộ, không ai có điểm. Nếu cả 4 thí sinh đều bị loại: → ở bất kỳ bước hàng nào mở toàn bộ. Cuối vòng, ← tắt đồ hoạ, rồi ← sang TT.

- **Tăng tốc**

4 câu, thời gian 20, 20, 30, 30 giây. Thuộc tính `seconds` trong `tangtoc.xml` ghi đè thời gian này. Người điều khiển gõ đáp án và thời gian trả lời của từng ghế trên màn quản lý.

Phím	Việc xảy ra	Âm thanh
← hoặc F3	Thẻ tên vòng hiện.	`tt_start`
←	Câu 1: thanh câu hỏi và khung ảnh hoặc video phóng to vào, kèm ánh loé. Video chưa chạy.	`tt_open_question`
→	Đồng hồ chạy; video bắt đầu chạy.	`tt_20s` / `tt_30s`
1-4	Đánh dấu hoặc bỏ đánh dấu ghế đúng (từ lúc câu hiện tới lúc chấm).	
→	Dừng đồng hồ, nhạc, video; khung câu hỏi tắt; cây đáp án hiện, xếp theo thời gian.	`tt_open_answers`
B	Xem lại khung câu hỏi (ảnh đáp án nếu XML có `<answerMedia>`). B lần nữa về cây. Khi đang xem lại, → bị khoá.	
→	Chấm. Ghế đúng xếp theo thời gian, nhận 40 / 30 / 20 / 10; nhánh sai chuyển xám.	`vcnv_row_correct` / `vcnv_row_wrong`
→	Cất cây đáp án, còn nền trống. Sau câu 4: đồ hoạ tắt hẳn.	
→	Sang câu kế.	`tt_open_question`
← ←	Sau câu 4: kết thúc vòng, rồi sang VĐ.	

Hai ghế bằng thời gian thì bằng điểm, và người kế nhận hạng kế tiếp chứ không bỏ hạng: ba ghế đúng với 1.25, 1.25 và 2.29 giây nhận 40, 40 và 30 điểm. Ghế đúng mà không có thời gian xếp sau mọi ghế có thời gian.

ẢNH CHỤP MÀN HÌNH #6

Tăng tốc: khung câu hỏi có ảnh và thanh thời gian.

Hình 11. Vòng Tăng tốc: khung câu hỏi và thanh thời gian.

- **Về đích**

Thí sinh lần lượt lên sân khấu theo điểm hiện tại, cao trước. Mỗi người chọn 3 câu, mỗi câu 20 hoặc 30 điểm.

Phím	Việc xảy ra	Âm thanh
← hoặc F4	Thẻ tên vòng hiện.	`vd_start`
←	Thứ tự thi tính theo điểm hiện tại, cao trước (bằng điểm: ghế số nhỏ trước). Người đầu lên sân khấu; chỉ có âm thanh.	`vd_ready`
←	Màn chọn gói hiện.	`vd_choose`
2 / 3	Chọn câu 20 / 30 điểm (3 lượt chọn, dấu ✓ vào ô tương ứng).	`vd_pick`
U	Bỏ lượt chọn cuối.	
← (đủ 3 lượt)	Màn chọn tắt; hai giây sau nhóm câu hỏi và hộp điểm phóng to vào, thanh câu hỏi để trống.	`vd_open_question` (phát ngay)
S	Đặt ngôi sao hy vọng cho câu chưa hiện (mỗi thí sinh một lần). Đặt được khi thanh câu hỏi trống, hoặc sau khi câu trước đã chấm xong.	`vd_star`
→	Hiện câu.	
→	Đồng hồ chạy: câu 20 điểm 15 giây, câu 30 điểm 20 giây.	`vd_15s` / `vd_20s`
↑	Đúng: cộng giá trị câu (nhân 2 nếu có ngôi sao).	`vd_correct`
↓	Sai: có ngôi sao thì trừ giá trị câu ngay. Không phát tiếng sai. Mở 5 giây giành quyền, thanh chạy ngược.	`vd_5s`
1-4	Ghế khác bấm giành quyền (người đầu tiên khoá). Thanh 5 giây và nhạc vẫn chạy tới hết.	`kd_buzz`
↑	Người giành đúng: cộng giá trị cho họ, trừ giá trị của người trên sân khấu (không trừ nếu có ngôi sao).	`vd_correct`
↓	Người giành sai: trừ nửa giá trị.	`vd_wrong`

→	Sang câu kế ngay (cũng dùng khi 5 giây giành quyền không có ai bấm).	
→ sau câu 3	Nhóm câu hỏi và hộp điểm tắt, còn nền trống.	`vd_turn_complete`
←	Người kế lên sân khấu (thứ tự tính lại theo điểm, trừ khi người điều khiển đã đặt tay). Sau người thứ 4: kết thúc vòng.	`vd_ready`
←	Sau người thứ 4: tắt đồ hoạ.	



Hình 12. Vòng Về đích: màn chọn gói.



Hình 13. Vòng Về đích: nhóm câu hỏi và ngôi sao hy vọng.

2.6 Luật tính điểm đang cài

Vòng	Tình huống	Điểm
KĐ lượt riêng	đúng / sai	+10 / 0
KĐ lượt chung	người bấm đúng / sai	+10 / −5
VCNV	mỗi ghế đúng một hàng (kể cả ô trung tâm)	+10, cộng lúc chấm hàng
VCNV, giải CNV	đúng khi đã mở 1 / 2 / 3 / 4 hàng	60 / 50 / 40 / 30
VCNV, giải CNV	đúng sau khi mở ô trung tâm	20
TT	ghế đúng xếp theo thời gian, hạng 1 / 2 / 3 / 4	40 / 30 / 20 / 10
VĐ	câu 20 / 30 điểm, trả lời đúng	+20 / +30, nhân 2 nếu có ngôi sao
VĐ	trả lời sai	0; có ngôi sao thì trừ giá trị câu
VĐ	giành quyền đúng	+ giá trị cho người giành; − giá trị cho người trên sân khấu (trừ khi có ngôi sao)
VĐ	giành quyền sai	− nửa giá trị (làm tròn)

- Điểm không bao giờ xuống dưới 0: khoản trừ lớn hơn số điểm đang có chỉ đưa điểm về 0.
- Tên hiển thị là hai chữ cuối của họ tên. Tên mặc định "Thí sinh N" hiện đủ.
- Tự đánh dấu đúng: so khớp không phân biệt hoa thường, dấu câu và khoảng trắng, nhưng có phân biệt dấu tiếng Việt ("ha noi" khác "Hà Nội"). Đáp án XML dạng "a / b" chấp nhận cả hai.
- Đánh dấu tự động bị gỡ nếu đáp án gõ bị sửa thành sai. Đánh dấu tay không bao giờ bị động tới.
- Điểm cộng, trừ nằm trong khối `Points` của `Variables.js`; cách đổi ở mục 3.3.

Chương 3

Hướng dẫn chỉnh sửa tính năng

Cho người mới sửa controller, biết JavaScript cơ bản. Mỗi kiểu chỉnh sửa có một ví dụ.

3.1 Chỉnh đồ hoạ

Toàn bộ đồ hoạ nằm trên một sân khấu cố định 1920×1080 px (`.stageContainerDiv`). Mọi div con trực tiếp của sân khấu đã có sẵn `position: absolute`; vị trí và kích thước từng phần tử khai báo bằng px trong `Styles/olympia.css`.

Muốn chỉnh	Sửa ở đâu
Vị trí, kích thước, màu, cỡ chữ	`Styles/olympia.css`
Hình nền các thanh và khung	Ảnh trong `O24/` (bảng dưới)
Chữ cố định vẽ bằng mã (ví dụ "CHƯỚNG NGẠI VẬT CÓ N CHỮ CÁI", "CÂU 20 ĐIỂM")	File `*Control.js` của vòng. Chữ gán bằng `textContent`, không dùng `innerHTML`.
Thanh thời gian	Cấu tạo ở Hình 14; chiều dài thanh là hằng trong file đồ hoạ (ví dụ `VCNV_TIMER_WIDTH = 839`), chấm đỏ là `TIMER_DOT = 18` trong `StageControl.js`.
Phần tử nào bị ẩn khi F6 mở	Quy tắc CSS ở Hình 15.

Ví dụ 3.1-a · Dời thanh câu hỏi Khởi động lên 40 px và phóng to chữ câu hỏi

Trong `Styles/olympia.css`, tìm luật `.questionBarDiv` (đang có `left: 100px; top: 715px;`) và luật `.questionTextP` (đang có `font-size: 32px;`). Trích từ mã, sau khi sửa:

```
.questionBarDiv {
        left: 100px;
        top: 675px;            /* trước: 715px */
        ...
}
.questionTextP {
        ...
        font-size: 36px;       /* trước: 32px - câu dài có thể tràn khung, thử với câu dài nhất của
đề */
}
```

Ví dụ 3.1-b · Thay ảnh thanh câu hỏi bằng ảnh khác

Mã tham chiếu ảnh theo đường dẫn, nên chép đè một file cùng tên là đổi được hình, không cần sửa mã. Ảnh mới nên cùng kích thước điểm ảnh với ảnh cũ (thanh câu hỏi KĐ là 1284×264), vì vị trí chữ và thanh thời gian được đặt theo kích thước đó. Giữ bản sao file cũ.

```
O24/Warm-up/question_bar.png                ← ảnh mới, 1284 × 264 px
O24/Warm-up/question_bar_without_timer.png  ← bản không có dải đồng hồ (lượt chung)
```

Muốn dùng tên file khác thì sửa đường dẫn trong mã, ví dụ bảng `QUESTION_BAR_IMAGES` ở đầu `Scripts/QuestionControl.js` và thẻ `<img>` tương ứng trong `default.html`.

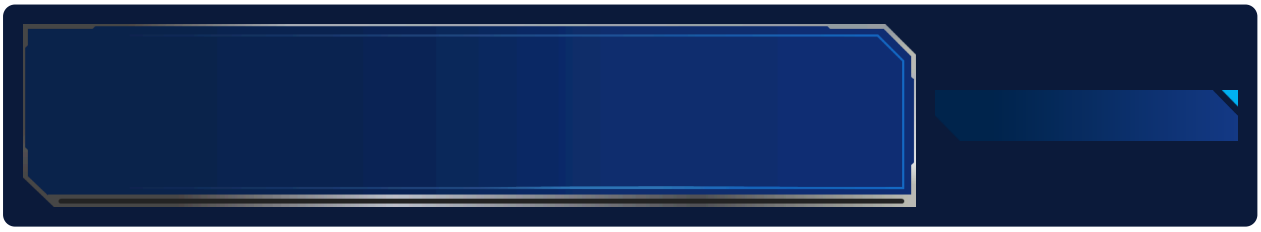
Ví dụ 3.1-c · Đổi chữ tiêu đề ô chữ VCNV

Trích từ `Scripts/VcnvControl.js`, hàm `obstacleTitleText()`. Sửa chuỗi trong hàm này là đổi tiêu đề ở mọi nơi dùng nó:

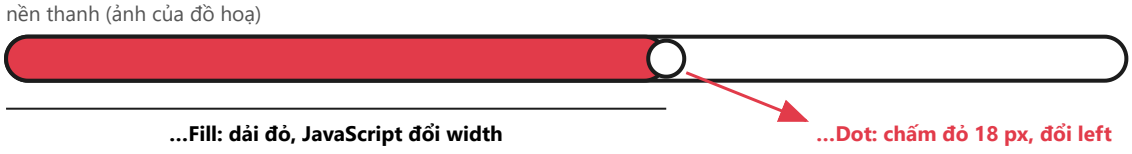
```
function obstacleTitleText() {
        var data = window.GameVariables.VcnvData || { keyword: '' };
        return 'CHƯỚNG NGẠI VẬT CÓ ' + window.GameVariables.vcnvLength(data.keyword) + ' CHỮ CÁI';
        /* ví dụ đổi thành: return 'TỪ KHOÁ GỒM ' + ... + ' KÝ TỰ'; */
}
```

Các ảnh thành phần đồ hoạ chính:

File	Thành phần
`O24/<vòng>/text.png`	Thẻ tên bốn vòng (Hình 1).
`O24/Warm-up/question_bar.png`, `question_bar_without_timer.png`	Thanh câu hỏi KĐ lượt riêng / lượt chung.
`O24/Warm-up/contestant_bar.png`, `contestant_answer_signal.png`	Thanh thí sinh thường / khi bấm chuông hoặc trên sân khấu.
`O24/Beat the Obstacle/Obstacle/*`	Ô chữ VCNV: thanh câu hỏi, ô tròn các trạng thái, khung ảnh 5 mảnh, số hàng, thẻ tên người xin CNV.
`O24/Beat the Obstacle/Answers/*`	Cây đáp án (dùng chung cho VCNV và TT).
`O24/Accelerate/Question/*`	Thanh câu hỏi và khung nội dung Tăng tốc.
`O24/Accelerate/Answers/contestant_name_bar.png`	Thẻ tên trên bảng tổng kết điểm (phím M).
`O24/Finish Line/*`	Màn chọn gói, ô tick, thẻ tên, thanh câu hỏi và hộp điểm Về đích.
`O24/Finish Line/star.webm`	Clip ngôi sao hy vọng. Tuỳ chọn: thiếu file thì chỉ không có hình ngôi sao.



Hình 13a. `O24/Warm-up/question_bar.png` và `O24/Warm-up/contestant_bar.png`.



width = phần thời gian đã trôi × chiều dài thanh. Ở Về đích, 5 giây giành quyền dùng cùng thanh nhưng chạy ngược (class reverse).

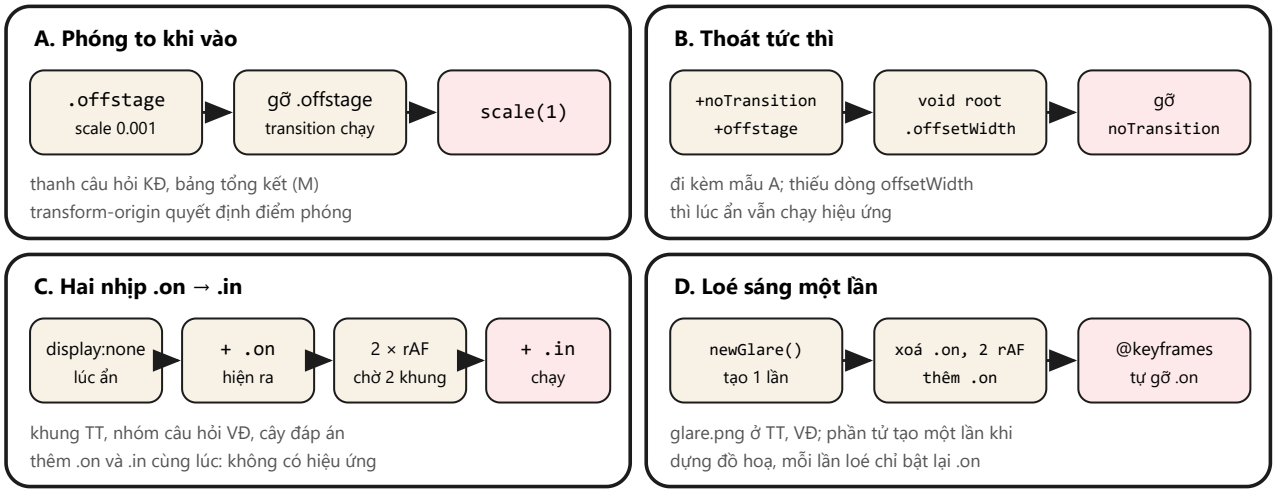
Hình 14. Cấu tạo thanh thời gian.



Hình 15. Con trực tiếp của sân khấu và phần tử nào bị F6 ẩn. Phần tử lồng sâu hơn không được quy tắc này ẩn.

3.2 Chỉnh animation

Nguyên tắc của dự án: animation viết bằng CSS, JavaScript chỉ bật và tắt class. Không thêm hiệu ứng mờ dần hoặc trượt ngoài những cái đã có, trừ khi người phụ trách đồ hoạ yêu cầu. Khi ẩn, phần tử thoát luôn tức thì. Có bốn mẫu (Hình 16); muốn đổi cách một phần tử xuất hiện, tìm xem nó dùng mẫu nào rồi sửa đúng phần của mẫu đó.



Hình 16. Bốn mẫu animation A, B, C, D.

- **Mẫu A: phóng to khi vào**

Phần tử luôn có `transition` trên `transform`. Trạng thái ẩn là class `.offstage`. Bỏ class thì phần tử phóng to về kích thước thật. Trích từ `olympia.css`:

```
.questionBarDiv {
        transform-origin: 0 0;             /* phóng từ góc trên trái */
        transform: scale(1);
        transition: transform 900ms cubic-bezier(0.16, 0.84, 0.44, 1);
}
.questionBarDiv.offstage { transform: scale(0.001); opacity: 0; }
.questionBarDiv.noTransition { transition: none; }
```

- **Mẫu B: thoát tức thì**

Trích từ `Scripts/QuestionControl.js`, hàm `hideQuestionBar()`:

```
root.classList.add('noTransition');
root.classList.add('offstage');
void root.offsetWidth;                 // ép trình duyệt áp dụng ngay, trước khi bật lại transition
root.classList.remove('noTransition');
```

- **Mẫu C: hai nhịp**

Dùng cho phần tử bị `display: none` lúc ẩn. Trích từ `Scripts/VcnvTreeControl.js`, hàm `zoomInAnswerTreePart()`:

```
part.classList.add('on');              // hiện ra, chưa có hiệu ứng
requestAnimationFrame(function () { requestAnimationFrame(function () { part.classList.add('in'); }); });
```

- **Mẫu D: loé sáng một lần**

`newGlare(class, parent)` trong `StageControl.js` tạo ảnh `O24/glare.png` một lần khi dựng đồ hoạ, và tự gỡ class `.on` khi hoạt ảnh `@keyframes` chạy xong. Mỗi lần muốn loé, mã xoá `.on`, chờ hai khung hình, rồi thêm lại `.on` (xem `showTangTocFrame()`). Đổi kiểu loé bằng cách sửa `@keyframes ttGlare` trong CSS.

Ví dụ 3.2-a · Đổi tốc độ và điểm phóng của thanh câu hỏi Khởi động (mẫu A)

```
.questionBarDiv {
        transform-origin: 50% 100%;        /* trước: 0 0 (góc trên trái); nay phóng từ giữa cạnh
dưới */
        transition: transform 550ms cubic-bezier(0.16, 0.84, 0.44, 1);   /* trước: 900ms */
}
```

Ví dụ 3.2-b · Đổi nhịp các thanh tên thí sinh hiện lần lượt

Trích từ `Scripts/ScoreboardControl.js`, hàm `revealContestantBars()`. Thanh ghế N hiện sau `300 + N × 190` ms. Đổi hai con số này là đổi nhịp:

```
window.GameVariables.ContestantBarRevealTimeouts.push(setTimeout(function () { ... }, 300 + seat *
190));
/* ví dụ: 4 thanh hiện cùng lúc sau 300 ms  →  300 + seat * 0 */
```

Bản thân thanh tên hiện không có hiệu ứng (`.contestantBar.shown { opacity: 1; }`); đó là lựa chọn có chủ ý của dự án.

Ví dụ 3.2-c · Rút ngắn khoảng chờ trước khi nhóm câu hỏi Về đích phóng vào

Trích từ đầu `Scripts/VeDichControl.js`:

```
var VD_GROUP_DELAY_MS = 2000;     /* bare background between the choose screen and the question
group */
/* ví dụ: 1000 để nhóm câu hỏi vào sau 1 giây */
```

Tương tự: `TREE_SPEED` đầu `VcnvTreeControl.js` (2 = cây đáp án chậm gấp đôi), `VD_CHOOSE_ZOOM_MS` (phải bằng thời lượng transition của `.vdChooseInner` trong CSS).

3.3 Chỉnh cơ chế có sẵn của vòng thi

- **Mức 1: không đụng logic**

Chỉ sửa cấu hình (khối *Initialization Variables* đầu `Variables.js`), file âm thanh hoặc file đề. Không viết thêm hàm.

Muốn đổi	Sửa ở	Ghi chú
Điểm	khối `Points`	`KD_WRONG_COMMON: -5` là điểm trừ khi sai ở lượt chung.
Thời gian	khối `Timings`	Nhạc đồng hồ vẫn dài như cũ; muốn khớp phải thay file nhạc.
Thay âm thanh	chép đè đúng tên cũ vào `Sounds/`	
Thêm âm thanh	`SoundMap`	Tự hiện trên bảng âm thanh. Nhãn tiếng Việt thêm vào `soundLabels` trong `ControlPanel.js`.
Nhạc nền lặp	`BedLoops`	Xem mục 5.7.
Đề và tên thí sinh	`Questions/*.xml`	Mỗi file có chú thích cách điền ở đầu.
Vị trí, kích thước đồ hoạ	`Styles/olympia.css`	Mục 3.1.

Ví dụ 3.3-a · Lượt chung KĐ sai bị trừ 10 điểm

```
KD_WRONG_COMMON: -10,                  /* trước: -5 */
```

Ví dụ 3.3-b · Câu 30 điểm Về đích được 25 giây, 5 giây giành quyền thành 7 giây

```
VE_DICH_STEAL: 7,                      /* trước: 5 */
VE_DICH_ANSWER: { 20: 15, 30: 25 }     /* trước: { 20: 15, 30: 20 } */
```

Thanh thời gian chạy đúng số giây mới. Nhạc `vd_20s.mp3`, `vd_5s.mp3` vẫn dài như cũ nên hết trước thanh.

Ví dụ 3.3-c · Thay tiếng "đúng" của Khởi động

Chép file mới vào `Sounds/`, đặt tên đúng `kd_correct.mp3` (ghi đè). Tải lại trang đồ hoạ. Nếu file mới là `.ogg`, phải đổi cả tên trong mã (tìm `'kd_correct.mp3'` trong `Scripts/Rounds/KhoiDong.js`) và khoá trong `SoundMap`.

Ví dụ 3.3-d · Thêm một tiếng mới vào bảng âm thanh của màn quản lý

```
// Scripts/Variables.js, trong SoundMap
'applause.mp3': true,                  /* tiếng vỗ tay, phát tay từ bảng âm thanh */

// Scripts/ControlPanel.js, trong soundLabels
'applause.mp3': 'Vỗ tay',
```

Chép `applause.mp3` vào `Sounds/`. Nút "Vỗ tay" tự xuất hiện; bấm nút thì phát trên kênh `StingerSound`, chồng lên các tiếng khác.

Ví dụ 3.3-e · Đổi đoạn lặp của nhạc nền Khởi động

```
gameVariables.BedLoops = {
        'kd_60s.mp3': { loopStart: 25, loopEnd: 50, releaseTo: 49, releaseSlack: 2 }   /* trước:
loopStart 30 */
};
```

Muốn một file nhạc khác có đoạn lặp: thêm một dòng cùng dạng với tên file đó, rồi phát nó bằng `startLongPassiveSound('ten.mp3')`.

Ví dụ 3.3-f · Soạn một câu Tăng tốc dùng video, 25 giây, có ảnh đáp án

```
<question seconds="25">
        <text>Đoạn video sau mô tả hiện tượng gì?</text>
        <media type="video">Questions/tt2.mp4</media>
        <answerMedia type="image">Questions/tt2_dapan.png</answerMedia>
        <answer>Nhật thực</answer>
</question>
```

`<answerMedia>` là ảnh hiện khi bấm B để xem lại. Đáp án "a / b" chấp nhận cả hai cách gõ. Không dùng HTML trong nội dung.

Xem biến trạng thái bằng Console

Mọi trạng thái là biến phẳng trong `window.GameVariables`. Mở F12 trên `default.html`, sang tab Console, gõ `window.GameVariables` để xem toàn bộ, hoặc một biến cụ thể như `window.GameVariables.VcnvSequenceCounter`. Lệnh `pressKey('ArrowLeft')` giả lập một lần bấm phím.

ẢNH CHỤP MÀN HÌNH #12

Console F12 của default.html khi gõ window.GameVariables.

Hình 17. Console của `default.html`.

- **Mức 2: có đụng logic**

Đổi âm thanh hoặc hình ở một bước

Sửa hàm bước trong `Scripts/Rounds/*.js`.

Ví dụ 3.3-g · Tăng tốc dùng tiếng chấm điểm riêng thay vì mượn của VCNV

Trích từ `Scripts/Rounds/TangToc.js`, cuối hàm `tangTocGradeQuestion()`:

```
/* trước */
startLongActiveSound(window.GameVariables.TangTocCorrectSeats.length ? 'vcnv_row_correct.mp3' :
'vcnv_row_wrong.mp3');
/* sau (nhớ thêm tt_correct.mp3, tt_wrong.mp3 vào Sounds/ và SoundMap) */
startLongActiveSound(window.GameVariables.TangTocCorrectSeats.length ? 'tt_correct.mp3' :
'tt_wrong.mp3');
```

Đổi phím

1. Trong `KeyboardController.js`, chuyển nhánh xử lý sang khối phím khác (`handleRoundKey` cho phím riêng vòng, `handleGlobalKey` cho phím chung).
2. Sửa bảng phím ở đầu file.
3. Sửa thuộc tính `data-key` của nút tương ứng trong `control.html`.

Tránh các phím đã dùng: M, Z, X, 0, ., F1-F6, và các phím vòng Q W E R U S B 5.

Ví dụ 3.3-h · Đặt ngôi sao hy vọng bằng phím N thay vì S

```
/* KeyboardController.js - xoá khối cũ: */
else if (lower == 's') // key 's' for the Ngôi sao hy vọng (VĐ)
{
        if (round == 'vd') { veDichPlaceStar(); return true; }
}
/* ...và thêm khối mới cùng chỗ: */
else if (lower == 'n') // key 'n' for the Ngôi sao hy vọng (VĐ)
{
        if (round == 'vd') { veDichPlaceStar(); return true; }
}

/* control.html - nút ngôi sao: */
<button type="button" class="bar-btn amber" id="vdStarBtn"><kbd>N</kbd>★ Đặt Ngôi sao hy
vọng</button>
```

Nút ngôi sao trên màn quản lý có `id="vdStarBtn"`; phím nó gửi nằm trong bảng `vdKeys` của `ControlPanel.js`: đổi `vdStarBtn: 's'` thành `vdStarBtn: 'n'`.

Đổi điều kiện của một bước

Ví dụ 3.3-i · Về đích: cho phép → đóng câu cả khi đã có người bấm giành quyền

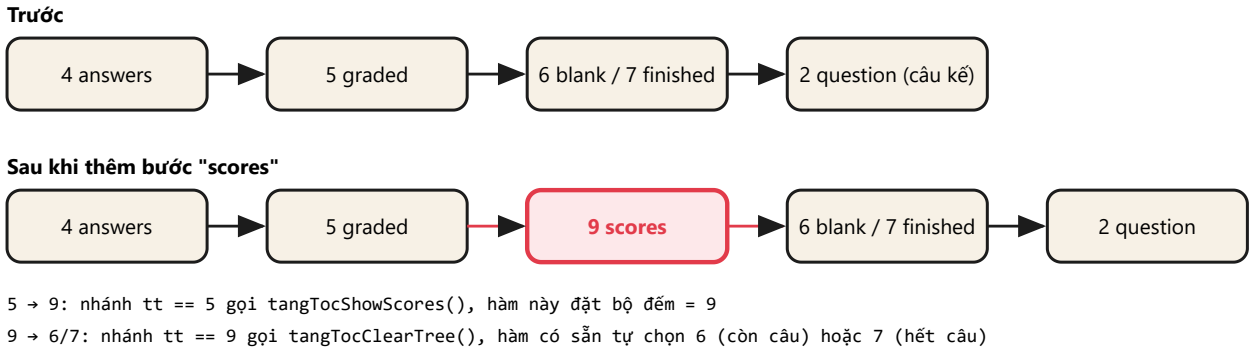
Trích từ `KeyboardController.js`, khối `ArrowRight`, nhánh `vd`:

```
/* trước: chỉ đóng câu khi chưa ai giành */
else if (vd == 7) { if (window.GameVariables.VeDichStealSeat === null) { veDichCloseQuestion(); } }
/* sau: bước 8 (đã có người giành, chưa chấm) cũng đóng được */
else if (vd == 7 || vd == 8) { veDichCloseQuestion(); }
```

Hệ quả: người điều khiển có thể bỏ qua việc chấm người giành. Kiểm tra lại bảng điểm sau khi đổi.

Thêm một bước vào vòng có sẵn

Quy tắc: không đánh số lại các bước cũ. Bước mới lấy số trống kế tiếp (Hình 18).



Hình 18. Chèn bước "scores" (số 9) giữa bước 5 và bước 6 / 7 của Tăng tốc mà không đổi số các bước cũ.

Ví dụ 3.3-j · Tăng tốc: thêm bước hiện riêng dải thanh điểm sau khi chấm

```
// 1. Scripts/Variables.js: thêm tên vào CUỐI mảng, rơi đúng chỉ số 9
gameVariables.TangTocPhaseNames = ['idle', 'title', 'question', 'timer', 'answers', 'graded',
'blank', 'finished', 'done', 'scores'];

// 2. Scripts/Rounds/TangToc.js: hàm bước mới
function tangTocShowScores() {
        hideAnswerTree();
        revealContestantBars();            /* 4 thanh tên + điểm hiện lần lượt */
        window.GameVariables.TangTocSequenceCounter = 9;
}

// 3. Scripts/KeyboardController.js, khối ArrowRight, nhánh round == 'tt'
else if (tt == 5) { tangTocShowScores(); }                          /* trước: tangTocClearTree(); */
else if (tt == 9) { hideContestantBars(); tangTocClearTree(); }
```

4. Tìm mọi chỗ dùng số bước của Tăng tốc (tìm `TangTocSequenceCounter` và `tt ==`): `tangTocCanMark()` (bước 2-4), khối phím B (bước 4, 5), `areRoundGraphicsOnScreen()` trong `ScoreSummaryControl.js` (coi bước từ 7 trở lên là xong vòng; bước 9 lớn hơn 7 nên phải sửa thành `tt == 7 || tt == 8`). 5. Tuỳ chọn: thêm `scores: 'Đang hiện điểm'` vào `ttPhaseText` trong `ControlPanel.js`.

Lưu ý: nếu bước mới dùng bảng F6 (`showBoardScreen()`) thì mọi phím bị khoá khi bảng mở, nên → ở bước mới sẽ không chạy. Ví dụ trên dùng thanh tên thay cho F6 vì lý do đó.

Chương 4

Hướng dẫn thêm tính năng

Cho người mới mở rộng controller. Mỗi kiểu thêm có một ví dụ; mã ví dụ chưa có trong dự án cho tới khi bạn tự thêm.

4.1 Thêm đồ hoạ và animation

Ví dụ 4.1-a · Dải chữ "GIẢI LAO" bật tắt bằng phím G (mẫu A + B)

Bước 1. `default.html` : thêm phần tử làm con trực tiếp của `.stageContainerDiv` (để F6 ẩn được nó), bắt đầu ở trạng thái ẩn.

```
<div class="breakDiv offstage"><p class="breakTextP">GIẢI LAO</p></div>
```

Bước 2. `Styles/olympia.css` : mẫu A. Không cần `position: absolute` vì sân khấu đã đặt cho mọi div con.

```
.breakDiv {
        left: 560px; top: 440px; width: 800px; height: 200px;
        transform-origin: 50% 50%;
        transform: scale(1);
        transition: transform 600ms linear;
}
.breakDiv.offstage { transform: scale(0.001); opacity: 0; }
.breakDiv.noTransition { transition: none; }               /* cần cho mẫu B */
.breakTextP { font-size: 96px; color: #fff; text-align: center; line-height: 200px; }
```

Bước 3. File mới `Scripts/BreakControl.js` , nạp trong `default.html` cạnh các `*Control.js` khác (trước `KeyboardController.js`):

```
function showBreakBanner() {
        var root = document.querySelector('.breakDiv');
        if (!root) { return; }
        root.classList.remove('offstage');
        window.GameVariables.IsBreakBannerShowing = true;
}

function hideBreakBanner() {
        var root = document.querySelector('.breakDiv');
        if (!root) { return; }
        root.classList.add('noTransition');
        root.classList.add('offstage');
        void root.offsetWidth;
        root.classList.remove('noTransition');
        window.GameVariables.IsBreakBannerShowing = false;
}

function toggleBreakBanner() {
        if (window.GameVariables.IsBreakBannerShowing) { hideBreakBanner(); } else {
showBreakBanner(); }
}
```

Bước 4. `Variables.js` , khối *Boolean Variables*: `gameVariables.IsBreakBannerShowing = false;`

Bước 5. `KeyboardController.js` , trong `handleGlobalKey` , và thêm dòng `G` vào bảng phím đầu file:

```
if (lower == 'g') { toggleBreakBanner(); return true; }              // key 'g' - "GIẢI LAO" banner
```

Muốn chặn phím `M` khi dải đang hiện: thêm `window.GameVariables.IsBreakBannerShowing ||` vào biểu thức `return` của `areRoundGraphicsOnScreen()` .



Hình 19. Vị trí dải "GIẢI LAO" trên sân khấu (minh hoạ).

Ví dụ 4.1-b · Thêm ánh loé khi dải "GIẢI LAO" hiện (mẫu D)

```
/* BreakControl.js - dựng một lần (gọi trong init() của Init.js, sau renderBoardScreen()) */
function renderBreakBanner() {
        var root = document.querySelector('.breakDiv');
        if (root && !root.querySelector('.breakGlareImg')) { newGlare('ttGlareImg breakGlareImg',
root); }
}

/* trong showBreakBanner(), sau khi gỡ .offstage */
var glare = root.querySelector('.breakGlareImg');
if (glare) {
        glare.classList.remove('on');
        requestAnimationFrame(function () { requestAnimationFrame(function () {
glare.classList.add('on'); }); });
}
```

Class `ttGlareImg` dùng lại hiệu ứng loé của Tăng tốc; vị trí ánh loé chỉnh bằng một luật `.breakGlareImg { left: ...; top: ...; }`.

Chọn mẫu animation cho phần tử mới:

Phần tử mới	Dùng mẫu
Hiện bằng phóng to, ẩn ngay	A cho lúc vào, B cho lúc ra
Ẩn bằng `display: none` (khung, nhóm, cây)	C: hai nhịp `.on` rồi `.in`
Chỉ cần một lần loé sáng	D: `newGlare(class, parent)`
Hiện ngay, không hiệu ứng (như thẻ tên vòng)	Chỉ bật tắt một class, CSS không có `transition`

4.2 Thêm cơ chế mới cho vòng thi

- **Phím mới**

Phím dùng ở mọi vòng: thêm vào `handleGlobalKey`. Phím riêng của một vòng: thêm khối vào `handleRoundKey`; khối trả `true` khi vòng đã dùng phím, `false` để phím rơi xuống `handleGlobalKey`. Phím có tác dụng mặc định của trình duyệt (phím F, dấu cách, mũi tên, Backspace) phải có trong `SWALLOWED_KEYS`.

Ví dụ 4.2-a · Phím chung P: dừng mọi âm thanh

```
/* KeyboardController.js, trong handleGlobalKey */
if (lower == 'p') { stopAllSounds(); return true; }                    // key 'p' - stop every sound
```

Ví dụ 4.2-b · Phím riêng VCNV A: mở toàn bộ ảnh và chữ (giống nút "Mở hết ảnh + đáp án")

```
/* KeyboardController.js, trong handleRoundKey, thêm một khối trước "return false;" cuối hàm */
else if (lower == 'a') // key 'a' for revealing the whole obstacle (VCNV)
{
        if (round == 'vcnv') {
                if (vcnv >= 2 && vcnv <= 10) { vcnvRevealEverything(); }
                return true;
        }
}
```

Ở các vòng khác, khối này trả `false` nên phím A rơi xuống phím chung (không có tác dụng).

- **Nút mới trên màn quản lý**

Nút làm việc của một phím có sẵn chỉ cần HTML trong `control.html`: thuộc tính `data-cmd` là lệnh phím của vòng (`kdKey`, `vcnvKey`, `ttKey`, `vdKey`), `data-key` là phím; `data-sendkey` gửi một phím chung.

Ví dụ 4.2-c · Nút "Đếm 3 giây" cho Khởi động (phím có sẵn, chỉ cần HTML)

```
<button type="button" class="bar-btn purple" data-cmd="kdKey" data-key=" "><kbd>␣</kbd>Đồng hồ 3
giây</button>
<button type="button" class="btn sm" data-sendkey="F6">F6 Bảng kết quả</button>
```

Ví dụ 4.2-d · Nút "Giải lao" gọi toggleBreakBanner() (việc mới, cần ba chỗ)

```
// 1. Scripts/PanelCommands.js
window.GameVariables.Commands.breakToggle = function () { toggleBreakBanner(); };

// 2. control.html, trong khu "Màn đồ hoạ"
<button type="button" class="btn sm" id="breakToggle">Giải lao</button>

// 3. Scripts/ControlPanel.js, cạnh các addEventListener('click', ...) khác trong hàm khởi tạo
$('#breakToggle').addEventListener('click', function () { gv.Sync.sendCommand('breakToggle', {});
});
```

Lệnh chạy trên `default.html`. Tham số thứ hai của `sendCommand` tới tay lệnh dưới tên `args`, ví dụ `sendCommand('setScore', { seat: 2, value: 40 })` → `args.seat`, `args.value`.

- **Thông tin mới trên màn quản lý**

Màn quản lý không đọc thẳng biến trong `Variables.js`; nó đọc bản chụp. Thông tin thuộc một vòng: thêm trường vào khối của vòng đó trong `buildRoundStateForPanel()` (`PanelStateControl.js`) rồi đọc trong hàm `render...` của vòng trong `ControlPanel.js`. Biến không thuộc vòng nào: thêm vào `buildSnapshot()` của `StateSync.js`, đọc trong `render(s)`.

Ví dụ 4.2-e · Màn quản lý cho biết dải "GIẢI LAO" đang hiện

```
// 1. Scripts/StateSync.js, trong buildSnapshot()
breakBanner: !!gv.IsBreakBannerShowing,

// 2. control.html, trong khu "Màn đồ hoạ"
<div class="kv"><span>Giải lao</span><b id="breakState">–</b></div>

// 3. Scripts/ControlPanel.js, trong function render(s)
$('#breakState').textContent = s.breakBanner ? 'HIỆN' : 'ẩn';
```

Ví dụ 4.2-f · Hiện số hàng VCNV đã mở trên tab VCNV

Trường `openedCount` đã có trong khối `vcnv` của `buildRoundStateForPanel()`, nên chỉ cần bước đọc. Trong hàm vẽ tab VCNV của `ControlPanel.js` (tìm `vcnv.phase`), thêm một dòng ghi `vcnv.openedCount` vào phần tử bạn tạo trong `control.html`. Trường chưa có thì thêm vào khối trước, ví dụ `prize: vcnvObstaclePrize(),`.

- **Âm thanh mới ở một bước**

Có sẵn ví dụ trong mã: `kd_complete.ogg` phát sau khi đồ hoạ tắt 1 giây (`khoiDongHideGraphicsThenComplete()`):

```
setTimeout(function () { startGeneralSound('kd_complete.ogg'); }, 1000);
```

Ví dụ 4.2-g · Phát tiếng khi hết giờ 15 giây của một hàng VCNV

```
/* Scripts/Rounds/VCNV.js, trong vcnvStartTimer(), hàm onDone của startClock */
onDone: function () {
        setObstacleTimer(1);
        window.GameVariables.IsVcnvTimerRunning = false;
        window.GameVariables.IsVcnvTimerDone = true;
        startGeneralSound('vcnv_timeup.mp3');            /* dòng thêm; nhớ khai báo trong SoundMap
*/
        window.GameVariables.changed();
}
```

- **Cơ chế tự động mới**

Ghi chú: thay đổi trạng thái bên trong `setTimeout`, `onDone` của đồng hồ hay sự kiện `ended` không đi qua `Keyboard.dispatch`, nên phải tự gọi `window.GameVariables.changed()` để màn quản lý cập nhật.

Ví dụ 4.2-h · VCNV: hết 15 giây thì cây đáp án tự hiện, không cần bấm →

```
/* Scripts/Rounds/VCNV.js, trong vcnvStartTimer(), cuối onDone */
onDone: function () {
        setObstacleTimer(1);
        window.GameVariables.IsVcnvTimerRunning = false;
        window.GameVariables.IsVcnvTimerDone = true;
        if (window.GameVariables.VcnvSequenceCounter == 5) { vcnvRevealAnswers(); }   /* dòng thêm
*/
        window.GameVariables.changed();
}
```

Điều kiện `== 5` giữ an toàn: nếu có người xin trả lời CNV lúc hết giờ (bộ đếm đang là 9), cây không tự hiện.

4.3 Thêm vòng thi: ví dụ Câu hỏi phụ

Vòng Câu hỏi phụ (mã `tb`) dùng phím F5 đã dành sẵn. Luật dưới đây đã được thống nhất với người dùng; mã chưa viết.

• Luật của vòng

- Chỉ dùng khi cả 4 thí sinh đã thi xong Về đích (`VeDichDone` có đủ 4 ghế).
- Màn quản lý phát hiện nhóm bằng điểm cao nhất và nhóm bằng điểm cao nhì (từ 2 người trở lên). Người điều khiển chọn nhóm cần phân định.
- Không đổi điểm, không có thẻ tên vòng. Chỉ hiện thanh tên của nhóm đang thi.

Phím	Việc xảy ra	Âm thanh
→	Hiện câu.	`tt_open_question`
Space	Đồng hồ 15 giây chạy liên tục (bấm chuông không dừng đồng hồ).	`vcnv_15s`
1-4	Bấm chuông. Chỉ ghế trong nhóm và chưa sai ở câu đó.	`kd_buzz`
↑	Đúng, dừng đồng hồ. Người đó xếp trên; những người còn lại tiếp tục thi với nhau.	`kd_correct`
↓	Sai, không phát tiếng: người đó mất quyền ở câu này, đồng hồ vẫn chạy cho người khác giành.	
tất cả sai hoặc hết giờ	Sang câu kế.	
còn một người	Phân định xong.	`vcnv_solved`

• Các chỗ phải thêm

File	Việc phải làm
`Scripts/Variables.js`	Bộ đếm `CauHoiPhuSequenceCounter` kèm bảng chú thích, mảng `CauHoiPhuPhaseNames`, biến trạng thái (tên bắt đầu bằng `CauHoiPhu`), `Timings.TB_BUZZ: 15`, `DataFiles.cauhoiphu`, biến chứa câu hỏi.
`Questions/cauhoiphu.xml`, `Scripts/Init.js`	File đề và hàm `getCauHoiPhuQuestions()`, gọi trong `getAllQuestionsForGame()`.
`Scripts/Rounds/CauHoiPhu.js`	Các hàm bước (tên bắt đầu bằng `cauHoiPhu`); thẻ `<script>` trong `default.html` cạnh các file vòng khác.

`Scripts/ScoreboardControl.js`	Hàm hiện riêng thanh tên của vài ghế (ví dụ `revealContestantBarsForSeats(seats)`).
`Scripts/KeyboardController.js`	`var tb = window.GameVariables.CauHoiPhuSequenceCounter;` đầu `handleRoundKey`, nhánh `round == 'tb'` trong từng khối phím cần dùng.
`Scripts/GeneralControl.js`	Dọn đồ hoạ trong `leaveRound`; tính nhóm hoà khi vào vòng trong `enterRound`.
`Scripts/PanelStateControl.js`	Khối `tb` trong `buildRoundStateForPanel()`.
`Scripts/PanelCommands.js`	Lệnh `tbKey` (gọi `dispatchTo('tb', args.key)`), `tbSelectGroup`, `tbReset`.
`control.html`, `Scripts/ControlPanel.js`	Tab và thanh nút của vòng (chép khối `data-tab="tt"` rồi đổi tên), hàm vẽ gọi từ `renderRound()`, đổi `tabForRound` từ `tb: 'vd'` thành `tb: 'tb'`, thêm nút `data-sendkey="F5"`.

- **Đề xuất các bước của bộ đếm**

Số	Tên đề xuất	Ý nghĩa
0	idle	Chưa chọn nhóm.
1	group	Đã chọn nhóm; thanh tên của nhóm hiện, thanh câu hỏi trống.
2	question	Câu hỏi đã hiện.
3	timer	Đồng hồ 15 giây chạy, chờ chuông (bấm chuông không đổi bước).
4	answered	Có người đúng, hoặc tất cả sai / hết giờ; → sang câu kế.
5	finished	Phân định xong, còn một người.
6	done	Tắt đồ hoạ.

Ví dụ 4.3-a · Khung nhánh 'tb' trong KeyboardController.js

```
/* đầu handleRoundKey, cạnh các biến kd, vcnv, tt, vd */
var tb = window.GameVariables.CauHoiPhuSequenceCounter;

/* trong khối if (key == 'ArrowRight') */
else if (round == 'tb') {
        if (tb == 1 || tb == 4) { cauHoiPhuShowQuestion(); }       /* hàm này đặt bộ đếm = 2 */
        else if (tb == 5) { cauHoiPhuFinish(); }                    /* đặt bộ đếm = 6 */
        return true;
}

/* trong khối else if (key == ' ') */
else if (round == 'tb') {
        if (tb == 2) { cauHoiPhuStartTimer(); }                     /* đặt bộ đếm = 3 */
        return true;
}

/* trong khối phím 1-4 */
else if (round == 'tb') {
        if (tb == 2 || tb == 3) { cauHoiPhuBuzz(seat); }            /* tự kiểm tra ghế thuộc nhóm,
chưa sai */
        return true;
}
```

Ví dụ 4.3-b · Hàm tính nhóm hoà (đặt cuối Variables.js, cạnh vdOrder)

```
/* [{ rank: 1|2, score, seats: [...] }] - chỉ nhóm từ 2 người trở lên */
gameVariables.tieGroups = function (players) {
        var scores = players.map(function (p) { return p.score; })
                .filter(function (s, i, a) { return a.indexOf(s) === i; })
                .sort(function (a, b) { return b - a; });
        var out = [];
        [0, 1].forEach(function (i) {
                if (scores[i] === undefined) { return; }
                var seats = players.filter(function (p) { return p.score === scores[i];
}).map(function (p) { return p.seat; });
                if (seats.length >= 2) { out.push({ rank: i + 1, score: scores[i], seats: seats });
}
        });
        return out;
};
```

Đặt trên `GameVariables` (không phải hàm toàn cục thường) để cả `ControlPanel.js` cũng gọi được, giống `ttRankPoints`.

4.4 Quy tắc an toàn khi thêm mã

- **Mọi file dùng chung một không gian tên.** Hai file cùng khai báo `function showBoard()` thì file nạp sau đè file nạp trước, không báo lỗi. Đặt tiền tố theo vòng (`vcnv...`) hoặc theo đồ hoạ (`showObstacleBoard`), hằng số viết hoa có tiền tố (`VCNV_TIMER_WIDTH`), và tìm tên trong thư mục `Scripts` trước khi đặt.
- `control.html` cũng nạp `Variables.js`, `StateSync.js` và `KeyboardController.js`, nhưng không nạp file đồ hoạ và âm thanh. Trong ba file này, không gọi hàm đồ hoạ ở cấp cao nhất (ngoài hàm).
- Không chạm DOM ở cấp cao nhất của file: lúc file được nạp, trang chưa dựng xong. Việc dựng đồ hoạ làm trong `init()`.
- Chữ lên màn hình dùng `textContent`, không dùng `innerHTML`.
- Hàm đồ hoạ mở đầu bằng `if (!root) { return; }`.
- Đồ hoạ mới là con trực tiếp của `.stageContainerDiv`.
- Thay đổi trạng thái trong `setTimeout`, `onDone` hay sự kiện `ended` phải tự gọi `window.GameVariables.changed()`.
- Đổi số bước của vòng thì tìm hết chỗ dùng số đó (ví dụ 3.3-j).

4.5 Kiểm tra và lỗi thường gặp

- Chạy `node --check Scripts/<file>.js` cho từng file đã sửa, nếu máy có Node.js. Không in gì là không có lỗi cú pháp.
- Mở F12 trên cả hai trang, tìm dòng lỗi màu đỏ (ghi tên file và số dòng).
- Chạy lại danh sách kiểm tra từng vòng, bằng cả phím lẫn nút trên màn quản lý.
- Trong Console dùng `pressKey('ArrowLeft')` để giả lập phím và xem `window.GameVariables.<Tên>SequenceCounter`.

Triệu chứng	Nguyên nhân thường gặp
`xxx is not defined`	Quên thẻ `<script>` của file mới, hoặc sai tên hàm, biến.

Phím không có tác dụng	Bộ đếm không ở bước mà nhánh chờ; bảng F6 đang mở; con trỏ đang ở trong ô nhập liệu.
Chức năng cũ chạy sai sau khi thêm hàm	Trùng tên hàm toàn cục.
Không có hiệu ứng	Thêm `.on` và `.in` cùng lúc, hoặc thiếu `transition` trong CSS.
Ẩn mà vẫn có hiệu ứng	Thiếu `void root.offsetWidth`, hoặc thiếu luật `.noTransition`.
Màn quản lý không cập nhật	Thiếu `changed()`, hoặc thiếu trường trong `PanelStateControl.js` / `buildSnapshot()`.
Âm thanh không phát	Sai tên file; chưa bấm phím nào trên trang đồ hoạ; file chưa có trong `Sounds/`.
Âm thanh phát hai lần	Đang mở hai tab `default.html`.

Chương 5

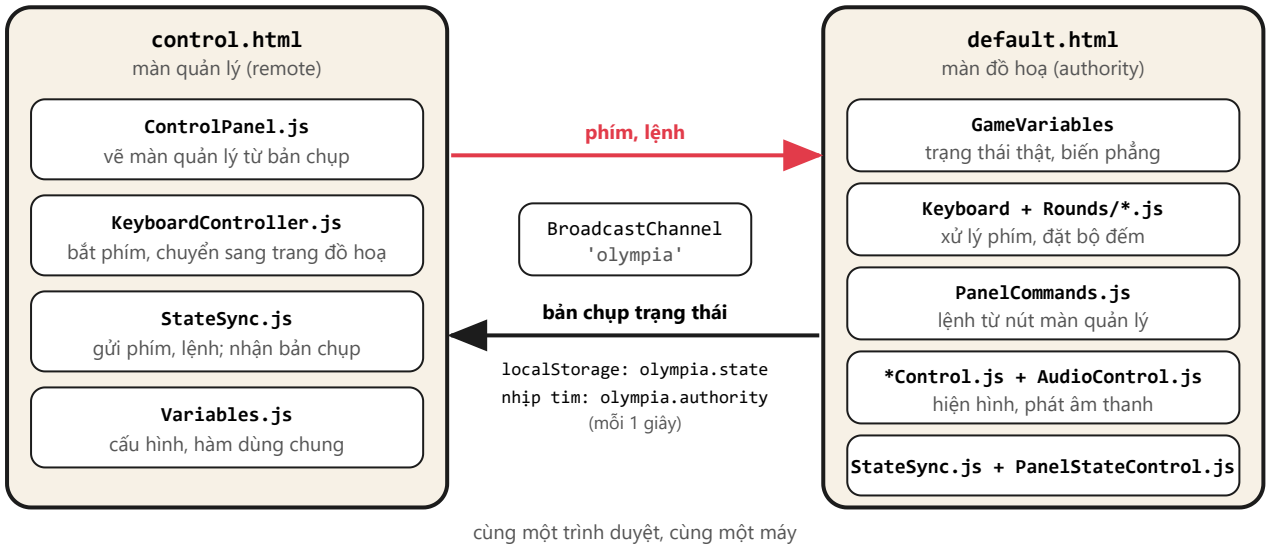
Cơ chế hoạt động chi tiết

Cho người sửa controller. Đọc sau khi đã biết cách dùng ở chương 2.

5.1 Kiến trúc hai trang

`default.html` là màn đồ hoạ, giữ trạng thái thật và là nơi duy nhất phát âm thanh. `control.html` là màn quản lý, không giữ trạng thái riêng mà vẽ lại từ bản chụp trạng thái nhận được.

Chiều từ màn quản lý sang màn đồ hoạ chỉ có phím và lệnh; chiều ngược lại chỉ có bản chụp trạng thái (Hình 20). Mọi thay đổi vì vậy đi qua đúng một đường xử lý trên trang đồ hoạ, dù người điều khiển bấm phím thật hay bấm nút.

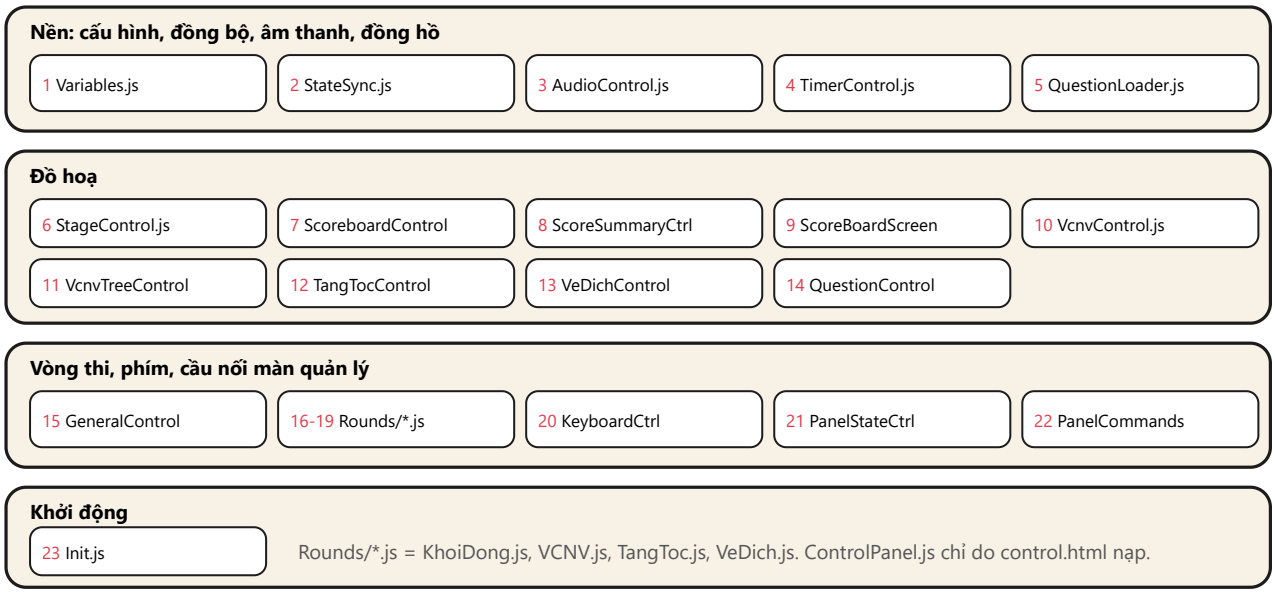


Hình 20. Kiến trúc hai trang.

Kênh truyền là **BroadcastChannel** tên `olympia`. Bản chụp cuối cùng còn được ghi vào **localStorage** khoá `olympia.state`, để màn quản lý mở sau vẫn có ngay trạng thái. Trang đồ hoạ ghi "nhịp tim" mỗi 1 giây vào khoá `olympia.authority`. Màn quản lý dùng nhịp tim để hiện trạng thái kết nối. Khi một trang đồ hoạ mới mở ra mà thấy nhịp tim của trang khác còn mới (dưới 3 giây), chính trang mới đó hiện cảnh báo đỏ "Một màn hình đồ hoạ khác đang mở".

5.2 Thứ tự nạp file

Mọi file nằm chung một không gian tên, nên hàm được gọi lúc chạy (khi bấm phím) có thể nằm ở file nạp trước hay nạp sau đều được. Chỉ mã chạy ngay khi nạp mới phụ thuộc thứ tự. `Init.js` nằm cuối vì `init()` gọi hàm dựng đồ hoạ của mọi file khác.



Hình 21. Thứ tự nạp trong `default.html`, đánh số 1 đến 23.

File	Chứa gì
`Variables.js`	Cấu hình (khối *Initialization Variables*) và mọi biến trạng thái, chia khối Sequence Counters, Boolean, Number, String, Array / Object, Audio, Interval / Timeout. Cuối file là hàm dùng chung: `createPlayer`, `getPlayer`, `displayName`, `vdOrder`, `ttRankPoints`, `normalizeAnswer`, `answerMatches`, `vcnvLength`.
`StateSync.js`	Đồng bộ hai trang: `buildSnapshot()`, `publish()`, `sendKey`, `sendCommand`, nhịp tim; định nghĩa `GameVariables.changed()`.
`AudioControl.js`	7 kênh âm thanh, nhạc nền có đoạn lặp.
`TimerControl.js`	Đồng hồ chung `startClock`, `stopClock`, `isClockRunning`.
`QuestionLoader.js`	Để trống; nội dung đã chuyển vào `Init.js`.
`StageControl.js`	Nền, thẻ tên vòng, hàm dựng phần tử `newElement`, `newImage`, `newGlare`, hằng `TIMER_DOT = 18`.
`ScoreboardControl.js`	4 thanh tên, `addScore`, `setScore`, chuông, người trên sân khấu, dải thanh tên (phím 0).
`ScoreSummaryControl.js`	Bảng tổng kết điểm (phím `M`), `areRoundGraphicsOnScreen()`.
`ScoreBoardScreenControl.js`	Bảng kết quả hiện tại (phím `F6`).
`VcnvControl.js`	Đồ hoạ VCNV: ảnh 5 mảnh, ô chữ 4 hàng, thanh câu hỏi, thẻ tên người xin trả lời CNV.
`VcnvTreeControl.js`	Cây đáp án, dùng chung cho VCNV và Tăng tốc.
`TangTocControl.js`	Khung câu hỏi Tăng tốc (ảnh hoặc video).
`VeDichControl.js`	Màn chọn gói, nhóm câu hỏi, hộp điểm, ngôi sao hy vọng.
`QuestionControl.js`	Thanh câu hỏi Khởi động, đồng hồ đếm 3-2-1.
`GeneralControl.js`	`switchRound()`, `leaveRound()`, `enterRound()`, `pressKey()`.
`Rounds/*.js`	Hàm bước của từng vòng; mỗi hàm tự đặt bộ đếm sang bước mới.
`KeyboardController.js`	Mọi phím: `handleRoundKey(key, round)`, `handleGlobalKey(key)`, và `GameVariables.Keyboard.dispatch / dispatchTo / switchRound / init`.

`PanelStateControl.js`	`buildRoundStateForPanel()`: đổi biến phẳng và bộ đếm thành object theo vòng, phase dạng chữ.
`PanelCommands.js`	24 lệnh từ nút màn quản lý (bảng ở mục 5.4).
`Init.js`	`init()`, `getContestantsForGame()`, `getAllQuestionsForGame()`, các hàm đọc XML, lưu tên gõ tay.
`ControlPanel.js`	Toàn bộ màn quản lý; chỉ `control.html` nạp.

5.3 Trạng thái và đồng bộ

Mọi trạng thái là biến phẳng trong `window.GameVariables`, tên bắt đầu bằng tên vòng: `VcnvCurrentRow`, `IsTangTocReviewing`, `VeDichSeat`... Không có biến ẩn trong closure, nên mở Console là thấy hết.

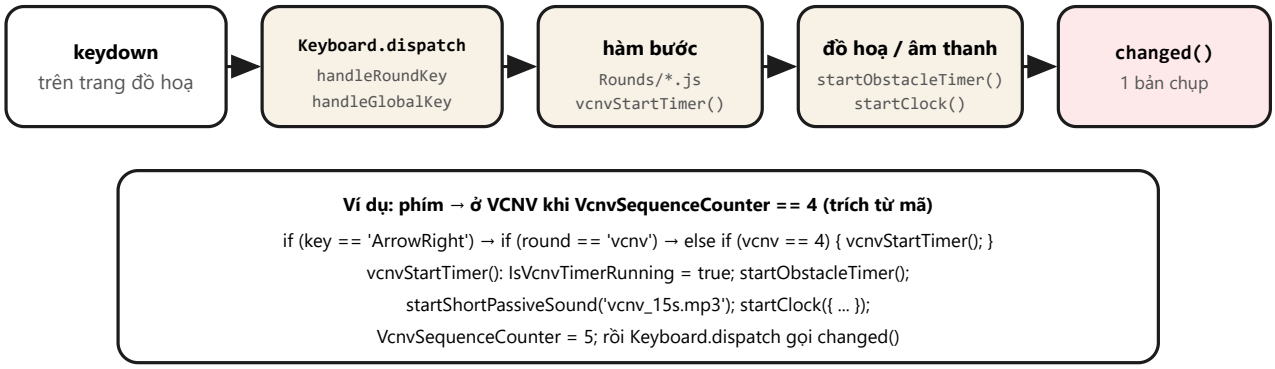
Mỗi khi trạng thái đổi, mã gọi `GameVariables.changed()`. Hàm này không gửi ngay: `StateSync.publish()` hẹn một lần gửi bằng `setTimeout(…, 0)`, nên nhiều lần gọi liên tiếp trong cùng một lần bấm phím chỉ tạo một bản chụp. Bản chụp (`buildSnapshot()`) gồm:

- danh sách người chơi (tên đầy đủ, tên hiển thị, điểm, cờ bấm chuông, trên sân khấu, bị loại VCNV, đã thi KĐ);
- vòng hiện tại, người đang hoạt động, thứ tự bấm chuông;
- `roundState`: kết quả của `buildRoundStateForPanel()`;
- nền, dải thanh tên, tổng kết điểm (`SummaryState`), bảng F6;
- đồng hồ (`TimerState`) và nhạc nền (`BedState`).

Bản chụp được ghi vào localStorage rồi gửi qua BroadcastChannel. Màn quản lý nhận và vẽ lại toàn bộ; đồng hồ trên màn quản lý tự đếm từ `TimerState` mà không cần nhận từng tích tắc.

5.4 Vòng đời một phím

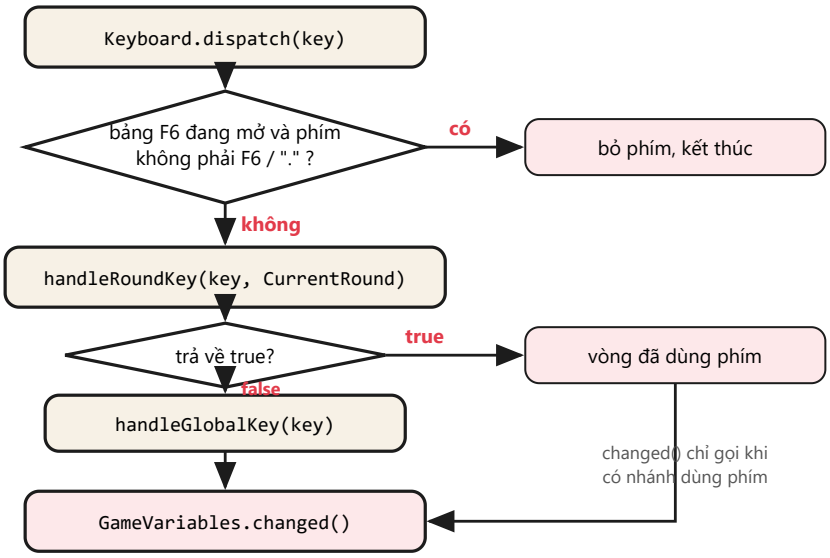
Một lần bấm phím đi qua ba tầng (Hình 22): tầng đầu bắt phím và chọn nhánh theo vòng và bước; tầng thứ hai là hàm bước, nơi đổi biến và đặt bộ đếm; tầng thứ ba là các hàm đồ hoạ, âm thanh và đồng hồ. Cuối cùng `changed()` báo cho màn quản lý.



Hình 22. Vòng đời một phím qua ba tầng.

- **Quyết định trong Keyboard.dispatch**

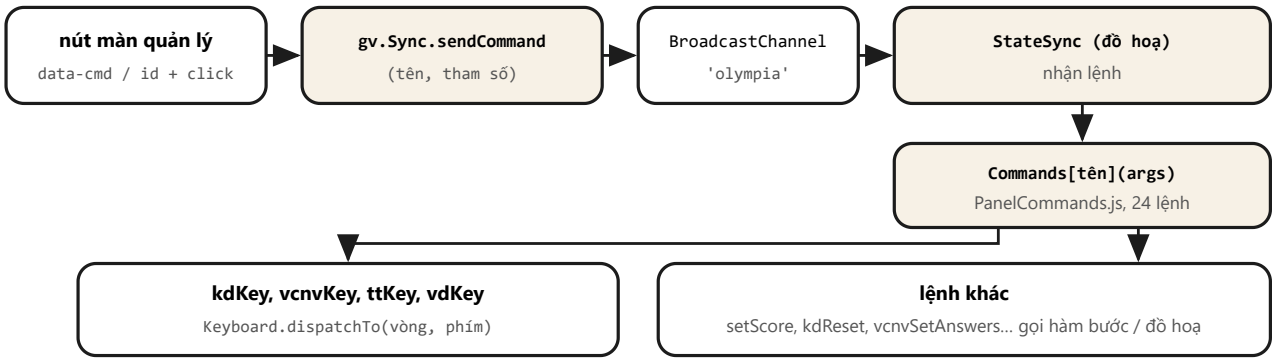
Trước đó, `init()` của bàn phím đã bỏ phím lặp (`e.repeat`), bỏ phím gõ trong ô nhập liệu và chặn tác dụng mặc định của các phím trong `SWALLOWED_KEYS`. Trên `control.html`, phím không vào `dispatch` mà được chuyển nguyên sang trang đồ hoạ bằng `Sync.sendKey`.



Hình 23. Quyết định trong `Keyboard.dispatch`. Lỗi trong một nhánh vòng được ghi ra Console và coi như phím đã dùng.

- **Đường đi của một lệnh từ nút**

Nút gọi `gv.Sync.sendCommand(tên, tham số)`. `StateSync` trên trang đồ hoạ gọi `GameVariables.Commands[tên](tham số)` (định nghĩa trong `PanelCommands.js`), rồi tự phát bản chụp mới. Các lệnh `*Key` gọi `Keyboard.dispatchTo(vòng, phím)`: chạy đúng nhánh vòng như phím thật, nhưng không rơi xuống phím chung (Hình 24).



Hình 24. Đường đi một lệnh từ nút trên màn quản lý. Sau lệnh, StateSync phát bản chụp mới về màn quản lý.

Nhóm	Lệnh (tên phải giữ nguyên, vì `ControlPanel.js` gửi đúng các tên này)
Âm thanh, điểm, tên	`playSound`, `stopSounds`, `setScore`, `setName`, `resetNames`
Chuyển phím vào vòng	`kdKey`, `vcnvKey`, `ttKey`, `vdKey`
Khởi động	`kdReset`, `kdResetCommon`, `kdClearBuzz`, `kdSetDone`
VCNV	`vcnvReset`, `vcnvReveal`, `vcnvFinish`, `vcnvUnEliminate`, `vcnvSetAnswers`
Tăng tốc	`ttSetAnswers`, `ttReset`, `ttGoto`
Về đích	`vdReset`, `vdGoto`, `vdSetOrder`

5.5 Bộ đếm bước của bốn vòng

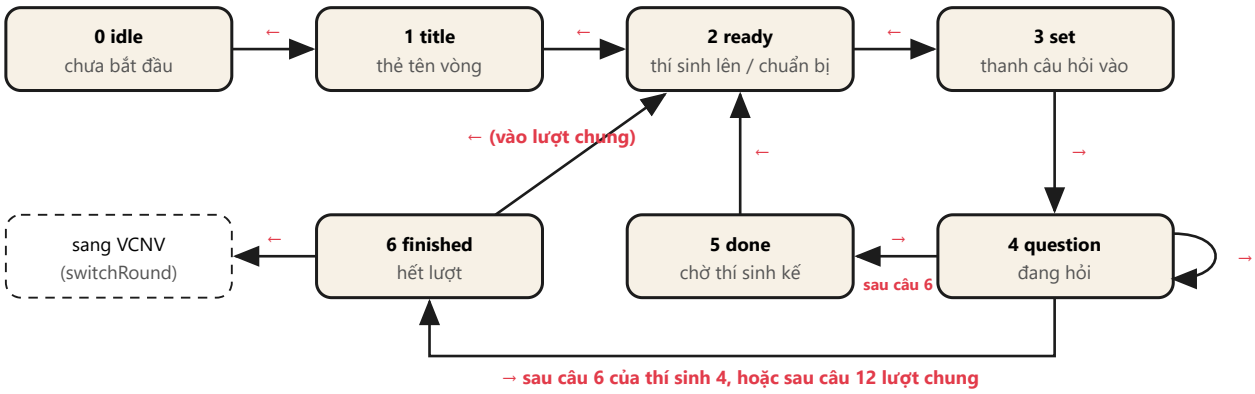
Mỗi vòng có một bộ đếm bước, cùng ý tưởng với `QuestionSequenceCounter` của WWTBAM gốc. Bộ đếm là số nguyên; mảng `…PhaseNames` cho biết tên của từng số, và màn quản lý in trạng thái theo tên đó. Bốn sơ đồ dưới

đây đã được đối chiếu với `KeyboardController.js` và `Rounds/*.js`.

Vòng	Bộ đếm	Các bước
KĐ	`KhoiDongSequenceCounter`	0 idle, 1 title, 2 ready, 3 set, 4 question, 5 done, 6 finished
VCNV	`VcnvSequenceCounter`	0 idle, 1 title, 2 board, 3 rowSelected, 4 rowQuestion, 5 rowTimer, 6 rowAnswers, 7 rowGraded, 8 rowBack, 9 guessing, 10 finalWindow, 11 finished, 12 done
TT	`TangTocSequenceCounter`	0 idle, 1 title, 2 question, 3 timer, 4 answers, 5 graded, 6 blank, 7 finished, 8 done
VĐ	`VeDichSequenceCounter`	0 idle, 1 title, 2 stageCall, 3 choose, 4 ready, 5 question, 6 timer, 7 steal, 8 stealAnswer, 9 answered, 10 blank, 11 finished, 12 done

- **Khởi động**

Hai lượt dùng chung một bộ đếm; biến `KhoiDongMode` (`'individual'` / `'common'`) cho biết đang ở lượt nào.

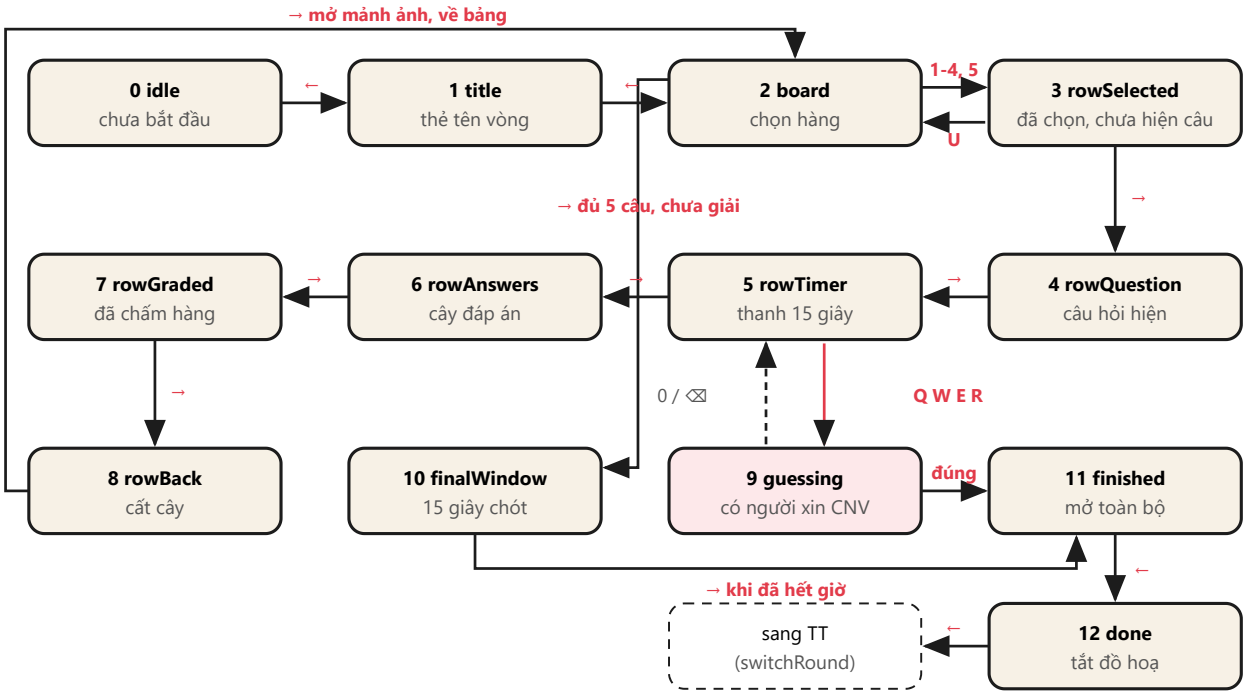


4 → 4: → sang câu kế. 6 → 2 chỉ khi đang ở lượt riêng; 6 → VCNV khi đang ở lượt chung. Lượt chung không có bước 1.

Hình 25. Sơ đồ trạng thái của `KhoiDongSequenceCounter`.

Từ	Đến	Phím	Hàm bước (trong `Rounds/KhoiDong.js`)
0	1	← (lượt riêng)	`khoiDongShowTitle()`
1, 5	2	←	`khoiDongCallNextContestant()`
2	3	←	`khoiDongOpenQuestionBar()` / lượt chung `khoiDongCommonOpenQuestionBar()`
3	4	→	`khoiDongShowQuestion(1)`
4	4	→	`khoiDongNextQuestion()`: câu kế
4	5 hoặc 6	→ sau câu cuối	lượt riêng `khoiDongCompleteTurn()` (6 nếu không còn ai); lượt chung `khoiDongCommonComplete()` → 6
6	2	← (lượt riêng)	`khoiDongEnterCommon()`: sang lượt chung
6	VCNV	← (lượt chung)	`switchRound('vcnv')`

- **Vượt chướng ngại vật**

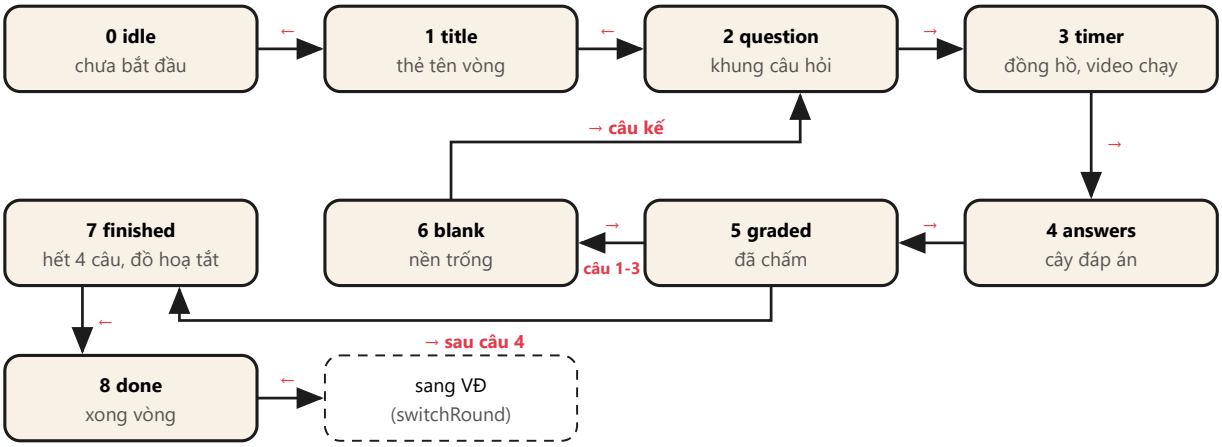


Hình 26. Sơ đồ trạng thái của `VcnvSequenceCounter`. Q W E R vào bước 9 được từ mọi bước 2-8 và 10 (sơ đồ vẽ một mũi tên đại diện từ bước 5); `0` hoặc ⌫ đưa về bước đang dở. Nếu cả 4 thí sinh bị loại, → ở các bước 2-8, 10 đi thẳng tới 11.

Từ	Đến	Phím	Hàm bước (trong `Rounds/VCNV.js`)
0	1	←, `F2`	`vcnvShowTitle()`
1	2	←	`vcnvShowBoard()`
2	3	`1`-`4`, `5`	`vcnvSelectRow(n)`; `U`: `vcnvUnselectRow()` về 2
3	4	→	`vcnvRevealQuestion()`
4	5	→	`vcnvStartTimer()`
5	6	→	`vcnvRevealAnswers()`. Ở bước 4, 5, 6: `1`-`4` gọi `vcnvMarkRowCorrect(seat)`
6	7	→	`vcnvGradeRow()`
7	8	→	`vcnvBackToBoard()`
8	2	→	`vcnvFinishRow()`: mở mảnh ảnh nếu hàng đúng
2	10	→ (đủ 5 câu, chưa giải)	`vcnvStartFinalWindow()`
10	11	→ sau khi hết giờ	`vcnvRevealEverything()`; bấm sớm thì không có tác dụng
2-8, 10	9	`Q` `W` `E` `R`	`vcnvSignal(seat)`; ghi bước cũ vào `VcnvPhaseBeforeGuess`
9	11	phím của ghế đã xin, hoặc `1`-`4`	`vcnvGuessCorrect(seat)`
9	bước cũ	`0`, hoặc ⌫ tới khi hết người xin	`vcnvGuessAllWrong()` / `vcnvUnsignal()`
11	12	←	`vcnvFinishRound()`

12	TT	←	`switchRound('tt')`

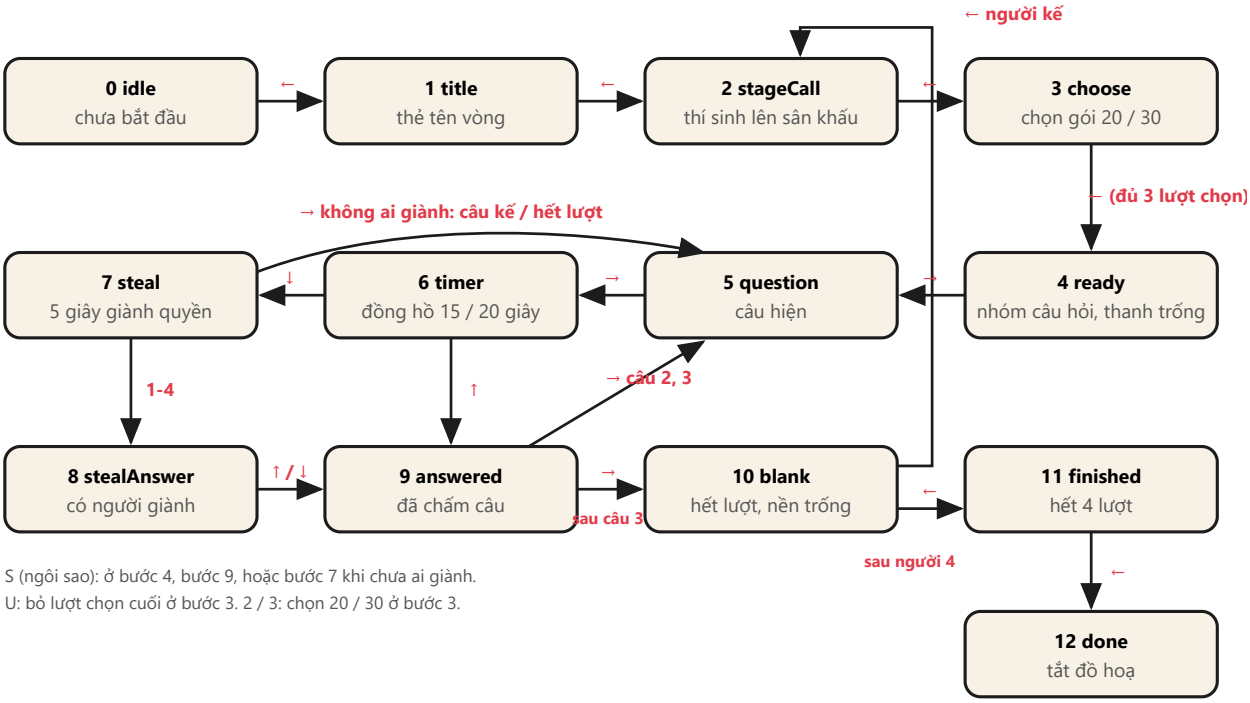
- **Tăng tốc**



Hình 27. Sơ đồ trạng thái của `TangTocSequenceCounter`. Phím B (bước 4, 5) chỉ bật cờ `IsTangTocReviewing`, không đổi bước; khi cờ bật, → bị khoá.

Từ	Đến	Phím	Hàm bước (trong `Rounds/TangToc.js`)
0	1	←, F3	`tangTocShowTitle()`
1	2	←	`tangTocShowFirstQuestion()`
2	3	→	`tangTocStartTimer()`. Ở bước 2, 3, 4: 1-4 gọi `tangTocMarkCorrect(seat)`
3	4	→	`tangTocRevealAnswers()`
4	5	→	`tangTocGradeQuestion()`
5	6 hoặc 7	→	`tangTocClearTree()`: 6 nếu còn câu, 7 sau câu 4
6	2	→	`tangTocNextQuestion()`
7	8	←	`tangTocFinishRound()`
8	VĐ	←	`switchRound('vd')`

- **Về đích**



Hình 28. Sơ đồ trạng thái của `VeDichSequenceCounter`. Từ bước 9 (và bước 7 khi không ai giành), → sang thẳng câu kế (bước 5); sau câu 3 thì sang bước 10.

Từ	Đến	Phím	Hàm bước (trong `Rounds/VeDich.js`)
0	1	←, F4	`veDichShowTitle()`
1	2	←	`veDichFirstTurn()`: tính thứ tự, gọi người đầu
2	3	←	`veDichOpenChooseScreen()`; ở bước 3, 2 / 3 gọi `veDichPick()`, U gọi `veDichUnpick()`
3	4	← (đủ 3 lượt)	`veDichPrepareFirstQuestion()`
4	5	→	`veDichRevealQuestion()`
5	6	→	`veDichStartTimer()`
6	9 / 7	↑ / ↓	`veDichAnswerCorrect()` / `veDichAnswerWrong()`
7	8	1-4	`veDichStealBuzz(seat)` (không nhận ghế đang trên sân khấu)
8	9	↑ / ↓	`veDichStealCorrect()` / `veDichStealWrong()`
9, 7	5 hoặc 10	→ (ở 7: chỉ khi chưa ai giành)	`veDichCloseQuestion()`: câu kế, hoặc bước 10 sau câu 3
10	2 hoặc 11	←	`veDichNextTurn()`: người kế, hoặc bước 11 sau người thứ 4
11	12	←	`veDichFinishRound()`
4, 9, 7	không đổi	S	`veDichPlaceStar()`

5.6 Chuyển vòng

`switchRound(id)` trong `GeneralControl.js` làm bốn việc theo thứ tự. Một, `leaveRound(cũ)` dọn đồ hoạ và dừng đồng hồ của vòng cũ; bộ đếm giữ nguyên, nên bấm phím F quay lại vẫn còn trạng thái (nhưng đồ hoạ của vòng không tự hiện lại). Hai, xoá chuông. Ba, đóng bảng tổng kết nếu đang mở. Bốn, `enterRound(mới)`: với VCNV, TT và VĐ chưa bắt đầu (bộ đếm = 0) thì thẻ tên hiện ngay; KĐ chờ phím ↵ đầu tiên.

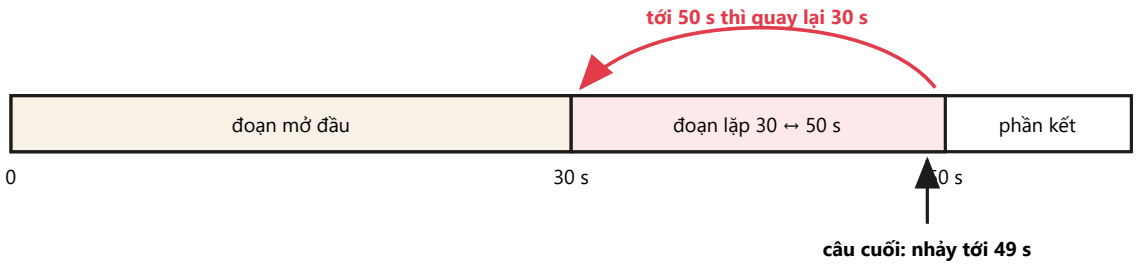
5.7 Hệ thống âm thanh 7 kênh

`AudioControl.js` có 7 kênh, mỗi kênh một `HTMLAudioElement` lưu trong `GameVariables` (ví dụ `GameVariables.TimerSound`). Một kênh chỉ phát một file: file mới cắt file cũ. Riêng các lời gọi có `overlap` (tham số thứ hai hoặc thứ ba bằng `true`) phát chồng và không bị `stop…()` chạm tới. Tên hàm giữ nguyên từ bộ WWTBAM gốc.

Hàm	Kênh	Dùng cho
`startGeneralSound(f)`	StingerSound	tiếng ngắn, luôn phát chồng
`startLongActiveSound(f, overlap)`	ResultSound	đúng, sai (Về đích phát chồng)
`startShortPassiveSound(f)`, `stopShortPassiveSound()`	TimerSound	nhạc đồng hồ một câu
`startLongPassiveSound(f)`, `releaseLongPassiveSound()`, `stopLongPassiveSound()`, `fadeOutLongPassiveSound(ms)`	RoundBedSound	nhạc nền cả lượt
`playBuzzerSound()`, `startShortActiveSound(f, overlap)`	BuzzerSound	chuông `kd_buzz`, tín hiệu CNV
`startLifelineActiveSound(f)`	StarSound	ngôi sao hy vọng
`startLifelinePassiveSound(f)`	AmbientSound	nhạc tổng kết
`stopAllSounds()`	tất cả	

- **Nhạc nền có đoạn lặp**

`kd_60s.mp3` chạy theo `BedLoops`: `loopStart 30`, `loopEnd 50`, `releaseTo 49`, `releaseSlack 2` (Hình 29). Tới giây 50, nhạc quay về giây 30. Khi câu cuối của lượt hiện, `releaseLongPassiveSound()` thoát vòng lặp và nhảy tới giây 49 để phát phần kết; cú nhảy bị bỏ qua nếu vị trí phát đã cách giây 49 dưới 2 giây. Vòng lặp được theo dõi bằng `setInterval` 40 ms; vị trí nhạc nền được gửi sang màn quản lý qua `BedState`.



Hình 29. Nhạc nền 3 đoạn của `kd_60s.mp3`. Độ dài phần kết theo file thật.

5.8 Đồng hồ chung

`TimerControl.js` chỉ có một đồng hồ, dựa trên `requestAnimationFrame`, với `startClock(opts)`, `stopClock()`, `isClockRunning()`. Gọi `startClock` mới sẽ huỷ đồng hồ đang chạy mà không gọi `onDone` của nó. `opts` gồm `mode` (`'down'`), `duration` (giây), `onTick(còn lại)`, `onDone()`, `label`. Thanh thời gian được vẽ trong `onTick` (Hình 14). Trạng thái đồng hồ nằm trong bản chụp dưới tên `TimerState`. Đồng hồ 3-2-1 của Khởi động cũng dùng đồng hồ chung này.

5.9 Khởi động trang

Hàm `init()` trong `Init.js` chạy khi trang tải xong (đầu file có cảnh báo "DO NOT TAMPER", giữ từ bộ gốc):

1. `getContestantsForGame()`: đọc `players.xml`; lỗi thì dùng "Thí sinh 1" đến "Thí sinh 4".
2. `getAllQuestionsForGame()`: đọc bốn file đề. File lỗi chỉ ghi lỗi ra Console; vòng đó hiện "(thiếu câu ...)".
3. Dựng mọi đồ hoạ rồi ẩn hết (`renderContestantBars()`, `renderObstacleBoard()`, `renderTangTocFrame()`, `renderVeDich()`, `renderBoardScreen()`...). Nền luôn hiện.
4. Bật đồng bộ ở vai authority; phát hiện trang đồ hoạ thứ hai.
5. Bật bàn phím.

Âm thanh chỉ phát sau phím bấm đầu tiên.

5.10 Đọc XML

`Init.js` đọc năm file trong `Questions/` bằng `fetch` bất đồng bộ và `DOMParser`. Bộ WWTBAM gốc đọc XML bằng `$.ajax` với `async: false`; Olympia không dùng jQuery. Kết quả được gán vào biến: `GameVariables.Players`, `KhoiDongQuestions`, `VcnvData`, `TangTocQuestions`, `VeDichQuestions`.

Tên gõ tay trên màn quản lý được lưu vào localStorage (khoá `olympia.names`) và được ưu tiên hơn `players.xml`. So khớp đáp án dùng `normalizeAnswer` và `answerMatches` ở cuối `Variables.js`; quy tắc ở mục 2.6. Chạy qua `file://` thì `fetch` bị trình duyệt chặn và mọi file XML đều lỗi.

Chương 6

So sánh với WWTBAM HTML5 Controller

Cho cả người điều khiển lẫn người sửa mã. Bộ gốc nằm trong thư mục WWTBAM Original Project.

6.1 Hai bộ khác nhau ở đâu

Bộ WWTBAM gốc là một trang, một người chơi, 15 cấp câu hỏi, điều khiển hoàn toàn bằng bàn phím trên chính trang đồ hoạ và thao tác DOM bằng jQuery `.transition()`. Olympia là hai trang đồng bộ, 4 người chơi, 4 vòng, DOM thuần (`querySelector`, `classList`) và animation viết bằng CSS.

Olympia giữ lối viết của bộ gốc: hàm toàn cục, biến phẳng trong `GameVariables`, tên hàm âm thanh, khuôn IIFE của `Variables.js`, cảnh báo "DO NOT TAMPER" trong `Init.js`. Khác biệt lớn nằm ở tầng phím và ở màn quản lý.



Hình 30. Màn đồ hoạ của bộ WWTBAM gốc.

6.2 Bảng đối chiếu file

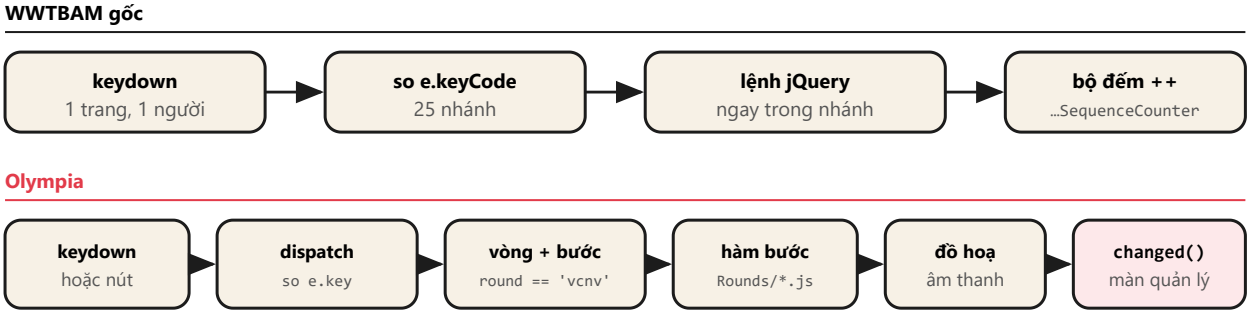
Số dòng đếm bằng `wc -l` trên hai thư mục `Scripts`. Olympia tổng 4927 dòng, trong đó `ControlPanel.js` 776 dòng. Bộ gốc tổng 4944 dòng, gồm cả jQuery nén.

WWTBAM gốc	Olympia	Ghi chú
`Variables.js` (196)	`Variables.js` (394)	Cùng khuôn IIFE, chia khối theo kiểu biến. Olympia thêm bảng bộ đếm và hàm dùng chung.
`KeyboardController.js` (1326), 25 nhánh `e.keyCode ==`	`KeyboardController.js` (303)	Gốc so `keyCode` và viết lệnh jQuery ngay trong nhánh. Olympia so `e.key`, thêm tầng "vòng", nhánh gọi hàm bước.

`QuestionSequenceCounter`, `MoneyTreeSequenceCounter`	4 bộ đếm, mỗi vòng một	
`AudioControl.js` (198)	`AudioControl.js` (180)	Giữ tên hàm gốc; thêm nhạc nền có đoạn lặp.
`TimerControl.js` (933)	`TimerControl.js` (60)	Olympia chỉ có một đồng hồ chung.
`Init.js` (285)	`Init.js` (183)	Cùng cảnh báo "DO NOT TAMPER". Gốc đọc XML bằng `$.ajax` với `async: false`; Olympia dùng `fetch` bất đồng bộ, đọc 5 file.
`GeneralControl.js` (89): logo, âm thanh theo cấp câu hỏi	`GeneralControl.js` (61): chuyển vòng	
`QuestionAndAnswerControl.js` (308)	`QuestionControl.js`, `TangTocControl.js`, `VeDichControl.js`, `VcnvControl.js`	Mỗi vòng một kiểu thanh câu hỏi.
`MoneyTreeControl.js` (357)	`ScoreboardControl.js`, `ScoreBoardScreenControl.js`	Thang tiền thành thanh điểm 4 người và bảng F6.
`WinningsControl.js` (139)	`ScoreSummaryControl.js`	Số tiền thắng thành tổng kết điểm.
`ContestantsControl.js` (22)	`getContestantsForGame()` trong `Init.js`	1 người chơi thành 4.
`FastestFingerControl.js` (144)	`VcnvTreeControl.js`	Nhiều người trả lời, xếp theo thời gian.
`LifelineControl.js` (911)	ngôi sao hy vọng trong `VeDichControl.js` và `Rounds/VeDich.js`	
`CameraControl.js`, `jquery-2.1.1.min.js`, `jquery.3d.js`, `accounting.js`	không có	Olympia không dùng jQuery.
không có	`StateSync.js`, `ControlPanel.js`, `PanelStateControl.js`, `PanelCommands.js`, `control.html`	Gốc chỉ điều khiển bằng bàn phím trên chính trang đồ hoạ. Olympia có màn quản lý thứ hai.

6.3 Luồng từ phím tới lệnh

Ở bộ gốc, một nhánh `keyCode` chứa luôn lệnh jQuery và tự tăng bộ đếm. Ở Olympia, nhánh phím chỉ chọn hàm bước theo vòng và bước; hàm bước gọi đồ hoạ, âm thanh và đặt bộ đếm; cuối cùng `changed()` báo cho màn quản lý (Hình 31). Muốn đổi việc xảy ra ở một bước: ở Olympia sửa hàm bước trong `Rounds/*.js`; ở bộ gốc sửa thẳng nhánh phím.



Hình 31. Luồng "phím → bộ đếm → lệnh" ở hai bộ.

6.4 Hình ảnh so sánh



Hình 32. Trên: thanh câu hỏi của bộ gốc (`WWTBAM Original Project/Images/Large_Strap_Unedited.png`). Dưới: thanh câu hỏi Khởi động của Olympia (`O24/Warm-up/question_bar.png`).

Chương 7

Một số lưu ý khác khi chạy

Cho người điều khiển buổi thi và người quản trị máy.

7.1 Máy chủ và trình duyệt

- Phải chạy qua IIS hoặc máy chủ web khác, không mở file bằng `file://` (trình duyệt chặn `fetch` đọc XML).
- Mọi đuôi file được phục vụ phải có trong `web.config`, thiếu sẽ báo lỗi 404.3. `web.config` đã khai báo font (woff, woff2, ttf, otf, eot), âm thanh (mp3, ogg, oga, wav), video (mp4, webm), và tắt bộ nhớ đệm (`DisableCache`) để bản sửa có hiệu lực ngay khi tải lại. Thêm loại file mới (ví dụ `.m4a`) thì phải thêm một dòng `<mimeMap>` tương ứng.
- Hai trang cùng một trình duyệt, cùng một máy, vì BroadcastChannel và localStorage không đi qua máy khác.
- Chỉ mở một tab `default.html`. Tab mở sau hiện cảnh báo đỏ và âm thanh sẽ lặp.
- Sân khấu cố định 1920×1080, không co giãn: màn phát 1920×1080, zoom 100 %, bấm F11.
- Trình duyệt chặn âm thanh cho đến khi trang đồ hoạ nhận một thao tác của người dùng. Bấm một phím trên `default.html` trước khi bắt đầu.

7.2 Tư liệu và dữ liệu

- Video Tăng tốc phát không tiếng (`muted`).
- Clip ngôi sao `O24/Finish Line/star.webm` là tuỳ chọn: trang kiểm tra file có tồn tại không trước khi nạp; thiếu file thì chỉ không có hình ngôi sao, tiếng `vd_star` vẫn phát.
- Thiếu hoặc lỗi file XML: vòng đó hiện "(thiếu câu ...)", trang không sập. Lỗi cú pháp XML xem ở Console.
- Tên gõ trên màn quản lý lưu trong localStorage của máy và được ưu tiên hơn `players.xml`. Đổi máy thì phải gõ lại, hoặc dùng nút "Về tên trong players.xml".
- Tải lại trang đồ hoạ là bắt đầu lại từ đầu: điểm và bộ đếm về 0 (chỉ tên gõ tay được giữ). Không tải lại giữa buổi thi; điểm có thể nhập lại tay ở khu Điểm số nếu cần.

7.3 Khi điều khiển

- F6 khoá mọi phím trừ F6 và dấu chấm. Phím M bị từ chối khi đồ hoạ vòng còn trên màn.
- Nút "Tất cả người xin đều sai, loại" (VCNV) không hỏi xác nhận. Bấm nhầm thì dùng "Khôi phục quyền thi cho ghế...".
- Không giữ phím: phím lặp bị bỏ qua.
- Phím gõ trong ô nhập liệu của màn quản lý không điều khiển buổi thi. Nếu phím không chạy, kiểm tra con trỏ có đang nằm trong một ô nhập không.
- Thứ tự thi Về đích tự tính lại theo điểm ở mỗi lượt, trừ khi người điều khiển đã đặt tay bằng "Đổi thứ tự thi...".
- Chuyển vòng bằng phím F giữa chừng: đồ hoạ vòng cũ tắt nhưng bộ đếm giữ nguyên; quay lại thì đồ hoạ không tự hiện lại. Nên đi hết vòng bằng ← / →, hoặc dùng nút "đặt lại" của vòng.

Phụ lục

Bảng tra nhanh

Bảng phím, bảng "muốn làm gì thì mở file nào", danh sách ví dụ.

A.1 Bảng tra nhanh phím

Vòng	Phím	Tác dụng
Chung	F1 đến F5	Chuyển vòng KĐ, VCNV, TT, VĐ, Câu hỏi phụ
Chung	M	Tổng kết điểm: mở, người kế, đóng
Chung	F6	Bảng kết quả bật, tắt; khoá mọi phím khác
Chung	.	Tiếng ô trống
Chung	0	Ẩn, hiện dải thanh tên
Chung	Z / X	+10 / −5 cho người đang hoạt động
KĐ	←	Thẻ tên; thí sinh kế; thanh câu hỏi; vào lượt chung; sang VCNV
KĐ	→	Câu 1; câu kế; hết lượt
KĐ	Space	Đếm 3-2-1
KĐ	↑ / ↓	Đúng / sai
KĐ chung	1-4	Bấm chuông
KĐ chung	Backspace	Xoá chuông bấm nhầm
VCNV	←	Thẻ tên; bảng ô chữ; tắt đồ hoạ; sang TT
VCNV	1-4	Chọn hàng (ở bảng); đánh dấu ghế đúng (câu, 15 giây, cây); ghế xin CNV đúng
VCNV	5	Chọn câu ô trung tâm
VCNV	U	Huỷ chọn hàng
VCNV	→	Hiện câu, 15 giây, cây đáp án, chấm, cất cây, mở ảnh; 15 giây chót; mở toàn bộ
VCNV	Q W E R	Ghế 1 đến 4 xin trả lời CNV; bấm lại là đúng
VCNV	Backspace	Bỏ người xin cuối
VCNV	0	Tất cả người xin đều sai, loại
TT	←	Thẻ tên; câu 1; kết thúc vòng; sang VĐ
TT	→	Đồng hồ, cây đáp án, chấm, cất cây, câu kế
TT	1-4	Đánh dấu đúng
TT	B	Xem lại khung câu hỏi / ảnh đáp án
VĐ	←	Thẻ tên; người đầu; màn chọn gói; nhóm câu hỏi; người kế; kết thúc; tắt đồ hoạ
VĐ	2 / 3	Chọn câu 20 / 30 điểm
VĐ	U	Bỏ lượt chọn cuối

VĐ	S	Đặt ngôi sao hy vọng
VĐ	→	Hiện câu, đồng hồ, câu kế / hết lượt
VĐ	↑ / ↓	Đúng / sai; chấm người giành quyền
VĐ	1-4	Giành quyền

A.2 Muốn làm gì thì mở file nào

Muốn làm	Mở file	Ví dụ
Đổi điểm, thời gian	`Scripts/Variables.js` (`Points`, `Timings`)	3.3-a, b
Đổi vị trí, kích thước, màu, cỡ chữ đồ hoạ	`Styles/olympia.css`	3.1-a
Đổi ảnh đồ hoạ	thư mục `O24/`	3.1-b
Đổi chữ cố định trên đồ hoạ	`Scripts/*Control.js`	3.1-c
Đổi hiệu ứng, tốc độ, nhịp	`Styles/olympia.css`, hằng đầu file `*Control.js`	3.2-a, b, c
Thay âm thanh	thư mục `Sounds/` (chép đè đúng tên)	3.3-c
Thêm âm thanh vào bảng âm thanh	`Variables.js` (`SoundMap`), `ControlPanel.js` (`soundLabels`)	3.3-d
Đổi nhạc nền lặp	`Variables.js` (`BedLoops`)	3.3-e
Đổi đề, tên thí sinh	`Questions/*.xml`	3.3-f
Đổi âm thanh / hình ở một bước	`Scripts/Rounds/*.js`	3.3-g
Đổi phím	`KeyboardController.js`, `control.html`, `ControlPanel.js`	3.3-h
Đổi điều kiện của bước	`KeyboardController.js`	3.3-i
Thêm bước vào vòng	`Variables.js`, `Rounds/*.js`, `KeyboardController.js`, `ControlPanel.js`	3.3-j
Thêm đồ hoạ mới có hiệu ứng	`default.html`, `olympia.css`, file `*Control.js` mới	4.1-a, b
Thêm phím	`KeyboardController.js`	4.2-a, b
Thêm nút trên màn quản lý	`control.html` (+ `PanelCommands.js`, `ControlPanel.js`)	4.2-c, d
Thêm thông tin trên màn quản lý	`PanelStateControl.js` / `StateSync.js`, `ControlPanel.js`	4.2-e, f
Thêm âm thanh ở một bước	`Rounds/*.js`	4.2-g
Thêm cơ chế tự động	`Rounds/*.js` (trong `onDone`, `setTimeout`)	4.2-h
Thêm vòng thi	xem mục 4.3	4.3-a, b
Đổi cách chuyển vòng	`Scripts/GeneralControl.js`	
Lỗi 404.3, file không tải được	`web.config`	

A.3 Danh sách ví dụ trong tài liệu

Mã	Nội dung	Mục
3.1-a	Dời thanh câu hỏi KĐ lên 40 px, phóng to chữ câu hỏi	3.1
3.1-b	Thay ảnh thanh câu hỏi	3.1
3.1-c	Đổi chữ tiêu đề ô chữ VCNV	3.1
3.2-a	Đổi tốc độ, điểm phóng thanh câu hỏi KĐ	3.2
3.2-b	Đổi nhịp các thanh tên hiện lần lượt	3.2
3.2-c	Rút ngắn khoảng chờ nhóm câu hỏi VĐ	3.2
3.3-a … f	Điểm, thời gian, thay âm thanh, thêm âm thanh, đoạn lặp, soạn câu TT video	3.3 mức 1
3.3-g … j	Tiếng chấm TT riêng, đổi phím S → N, đổi điều kiện đóng câu VĐ, thêm bước "scores" cho TT	3.3 mức 2
4.1-a, b	Dải "GIẢI LAO" (mẫu A + B), thêm ánh loé (mẫu D)	4.1
4.2-a … h	Phím P dừng âm, phím A mở hết VCNV, nút chỉ cần HTML, nút "Giải lao", báo trạng thái lên màn quản lý, hiện số hàng đã mở, tiếng hết giờ VCNV, tự hiện cây khi hết giờ	4.2
4.3-a, b	Khung nhánh 'tb' trong bàn phím, hàm tính nhóm hoà	4.3